Appendices to: Existence Before Relevance: Same-Category Demonstrations Narrow a Prerequisite-Chain Gap in LLM Plan Generation, and Pass On Their Defects

Zixuan Wang[1] **Ziming Dai**[2] **Yu Wu**[3] **Li Ning**[4]
[1]Shenzhen College of International Education [2]BASIS International Schools
[3]CuliuTech [4]Stellaris AI Limited
zw6963470@gmail.com benjamin.dai66@gmail.com
wuyu@culiu-tech.com lining@stellaris-ai.com

How to Read These Appendices

Relation to the main paper. These appendices accompany the main paper of the same title. Sections numbered with digits, the first two tables, the pipeline figure and the scoring equation point to that paper. Table, figure and equation numbers here continue from it.

No parameter updates. Every condition reported anywhere in this paper is inference-time only: the model endpoint and its weights are fixed, and only the input prompt is rebuilt (Appendix B). The files named `*_examples.json` under `data/training/` in the code release are *frozen demonstrations used as prompt data*, not examples used to train or fine-tune any evaluated model. No optimiser, gradient update, fine-tuning pass or weight persistence occurs at any point in the pipeline.

Confirmatory versus exploratory. The confirmatory evidence base is the frozen 60-task benchmark evaluated under the fixed static evaluator of §4.3, with the pre-specified 13-test Bonferroni family of Appendix G (α'=0.0038). Several appendices below instead report an *earlier* benchmark version (35 or 42 tasks), a *mismatched* weight configuration, *additional* sub-frontier models outside the pre-specified family, or an evaluator version predating the `setRecipeResolver` fix. Each of those carries **Exploratory** in its heading and its own caveat in its opening line. They are retained for transparency and to motivate future ablations; *no claim in the main text rests on them*, and they are never pooled with the confirmatory numbers. Where an appendix figure disagrees with a main-text figure, the main text governs, and Appendix N names the run that produced it.

The last appendix carries no claim. Appendix W reports a two-bot live study of a second retrieval stage of the same enhancer. It is kept so that the released system can be read end to end, exercises none of the components the main study varies, and can be skipped by a reader auditing the paper's claims.

A Scorer Cross-Validation against the Plancraft Oracle

Cross-validation against Plancraft oracle. We cross-validate the ingredient-feasibility check against the independent, published Plancraft oracle planner [Dagan et al., 2025]. The comparison is pinned to the 16 runs behind the primary-study rows of Table 1, all of Table 2 and the per-model BM25 rows of Table 12 — five models × {`no_retrieval`, `bm25`, `saer_cold`}, plus Qwen2.5-Max's `random` arm, 48 seed files. The Gemma4 replication runs are not included. Across their 11,177 `craftRecipe` invocations the two validators agree on **95.8%** (10,711/11,177; Cohen's κ=0.753 [Landis and Koch,

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Neuro-Symbolic Embodied Intelligence.

1977]), with 427 validator-lenient and 39 validator-strict disagreements. Selection is by run identity, not by file date: an earlier revision of this script took "the most recent seed files" from the results directory, which never looked inside the two subdirectories holding the primary runs and re-targeted itself every time a later ablation was added. The figure above is the pinned one, reproducible with `node scripts/eval/plancraft_crossval.mjs`. The oracle is the published `plancraft` package (v0.3.34) called through a local batch shim, not a reimplementation of it, so the comparison is against the authors' own planner rather than our reading of it.

Per-condition false positives. Table 3 splits the disagreements by condition. Homogeneity is not rejected ($\chi^2(3)$=4.78, p=0.189, smallest expected count 27.7), and the per-invocation ordering — most lenient on `no_retrieval` (4.47%), least on `saer_cold` (3.46%) — does *not* license the reading that residual leniency favours the baseline arm. Strict-pass is a plan-level predicate: one leniently credited craft can carry a whole plan, so the quantity that matters is lenient credits *per plan*, and the arms differ in how many crafts a plan contains. Across the 2,880 plans in these 48 files the retrieval arms issue about half again as many craft calls each (4.0–4.5 against `no_retrieval`'s 2.9), and the two effects multiply out the other way: 0.130 leniently credited crafts per plan on `no_retrieval` against 0.154 on `saer_cold`, 0.157 on `bm25` and 0.167 on `random`. We therefore claim no direction from this table. It bounds recipe-level disagreement; whether scorer leniency can account for the reported margins is answered instead by Appendix U, which charges the entire material budget — a far larger change than repairing these disagreements — and finds the existence effect and all five per-model effects still positive. The scorer operates on deterministic game data, independent of SAER's scoring function.

Table 3: Plancraft cross-validation by condition, over the pinned 48 seed files behind the primary-study rows of Table 1, Table 2 and Table 12. FP = validator passes a craft the oracle calls unsolvable (leniency); FN = the reverse; FP/inv. = FP as a share of all invocations, not of oracle-unsolvable ones. Reproduce with `node scripts/eval/plancraft_crossval_stratified.mjs`.

Condition	Invocations	Agree	FP	FP/inv.	FN
`no_retrieval`	2,619	95.1%	117	4.47%	11
`random`	724	94.9%	30	4.14%	7
`bm25`	3,815	95.9%	141	3.70%	15
`saer_cold`	4,019	96.4%	139	3.46%	6

What this agreement does and does not cover. The cross-validation walks each plan accumulating a virtual inventory that, like the scorer's, is credited but never debited, and puts a *per-invocation* question to the oracle: given this inventory state, is this one craft solvable? Both sides therefore receive the same non-consuming state, so the procedure cannot by construction detect a whole-plan double-spend — the disagreements it can find are recipe-level, not budget-level. The 95.8% should accordingly be read as "the two agree on single-craft feasibility given a state", not as "the scored plans are materially executable". It also covers one of the scorer's six issue classes. The other five — tool, smelt, precondition, naming and goal — have no external validation here, and the tool class is the one that dominates the failures of GPT-4o and Qwen2.5-Max (Table 21). The aggregate material budget is a separate question, which the scorer deliberately does not enforce (Appendix T) and which Appendix U takes up for the demonstration bank.

B SAER Prompt Assembly and the No-Update Property

The four component scores are defined in §3.3 (ingredient overlap I, keyword overlap K, mining-tier gap M, level proximity L) and are not repeated here. What the score is *used for* is stated here, because that is both the point at which the symbolic retriever hands off to the neural planner and the point at which the claim "no parameter updates" becomes checkable.

Let q be the request, x the agent state, c=`getIntent`(q) the inferred category and D_c the corresponding demonstration set. After scoring, the enhancer builds the base prompt

$$P_{\text{base}}(q, x) = \text{concat}\big(R,\ \text{Top}_k(D_c; S),\ q\big), \tag{2}$$

where R is the command and rule documentation, Top_k supplies the k=3 highest-ranked demonstrations, and the unchanged model returns $\hat{y}_0 = M_\theta(P_{\text{base}})$ with $\theta_{\text{after}} = \theta_{\text{before}}$. Selected examples

are serialised in descending score order, so the strongest match comes first within the demonstration block, which itself follows the command reference and precedes the task statement.

The text R is the same for every category and condition, and it is not bare. It lists the tool recipe patterns, the pickaxe tier each ore needs, a generic cold-start tool chain that ends at mining iron ore, the raw-ore drop names and the output format. So `no_retrieval` measures planning from these rules with no retrieved demonstration, not a model given only the task.

Every condition in this paper differs only in which D_c and which ranking populate Top_k: `no_retrieval` sets it empty, `random_retrieval` draws uniformly, `bm25` ranks lexically, and `saer_cold` ranks by S. Nothing else in Equation 2 varies, and no term in it is learned.

C Execution Modules

The validated step array is executed by **TaskManager**, which maintains a last-in-first-out (LIFO) task stack with three-layer error recovery (transient failures, missing prerequisites, semantic errors). **BlockNameResolver** canonicalises fuzzy block names to official Minecraft 1.20.4 identifiers via environment scan, `minecraft-data` lookup, and hardcoded fallback. These modules operate at runtime only and are not exercised by the static evaluation in §4.3.

D Evaluated Models

Table 4 lists the seven evaluated models with their scale, serving organisation, the exact API identifier each result was produced against (as recorded in the run files) and the sampling temperature the harness sent. The run files record the model identifier but no sampling parameter, so the temperature column is read from the harness: the evaluation profiles of the first three models set `temperature: 0.7`; the profiles used for the other four set none, and the model clients add no default, so those requests carried no temperature and the endpoint's own default applied. Every within-model comparison holds the model, and hence this setting, fixed; the three cross-model tests of the pre-specified family compare models run under different settings.

Table 4: Evaluated models. Primary five-model study: `no_retrieval` + `saer_cold` (cold-start, γ=0). Replication panel (Gemma4-26B-A4B, Gemma4-31B): `no_retrieval` + `saer_cold` + `saer_full` (the latter warm-start, generated before the recipe-resolver fix of §4.3, and *not* directly comparable; the `saer_cold` arm is, Appendix G), plus `bm25_retrieval` for the per-model BM25 comparison (Appendix O). †: extended thinking disabled (`enable_thinking=false`); thinking-mode ablation in Appendix Q tests whether enabling it lifts the no-retrieval L2 floor. ‡: also the four-condition-ablation model (§5.2). Temperature *default*: no temperature was sent, so the endpoint's own default applied.

Model	Size / Org	API identifier	Temperature
Qwen3-8B† [Qwen Team, 2025]	8B, Alibaba	`qwen3-8b`	0.7
GPT-4o [OpenAI, 2024]	—, OpenAI	`gpt-4o-2024-11-20`	0.7
Claude S. 4.5 [Anthropic, 2025]	—, Anthropic	`claude-sonnet-4-5-20250929`	0.7
Qwen2.5-Max‡ [Qwen Team, 2024]	100B+ MoE, Alibaba	`qwen-max`	default
DeepSeek-V3 [DeepSeek-AI, 2024]	671B MoE, DeepSeek	`deepseek-chat`	default
Gemma4-26B-A4B [Gemma Team, 2026]	26B (4B active), Google	`gemma-4-26b-a4b-it`	default
Gemma4-31B [Gemma Team, 2026]	31B, Google	`gemma-4-31b-it`	default

E Training Data

The demonstration pool consists of **25 crafting examples**, **21 collecting examples**, and **9 combat examples** (55 total), built to span L1–L3 difficulty. We wrote a small seed set by hand to fix the schema, an AI assistant drafted the remaining examples in that schema, and we checked each example by hand; Appendix U reports what that check missed. Each example follows a step-by-step schema with explicit `preconditions` and `produces` fields. L2 crafting examples each include the full

wooden→stone→iron pickaxe acquisition chain as prerequisite steps. Combat examples are retrieved only for the five combat tasks, in the main runs and in the combat probe (§4.4); category filtering keeps them out of the crafting and collecting tasks. As a follow-up check, we additionally audited all 55 examples command-by-command from an empty cold-start inventory, verifying every declared item-level precondition is satisfied by an earlier step's own output; this surfaced 7 residual gaps beyond the combat-pool fix of §4.4 (2 crafting, 4 collecting, 1 combat — e.g. a tool never crafted before its use, or a byproduct consumed without being declared), each corrected by prepending the missing acquisition chain or declaring the undocumented byproduct.

Bank revision timeline. Four appendices touch the bank's history, so we give the single ordering they all refer to. **(i) Before the freeze (2026-05-30).** Six L2 crafting demonstrations (`iron_axe`, `iron_pickaxe`, `iron_shovel`, `iron_hoe`, `shears`, `shield`) declared a stone pickaxe in step 2 that no earlier step crafted. They were repaired on the superseded 35-task benchmark, before the 60-task benchmark was frozen and before the earliest run reported here; the run timestamps in Appendix N all postdate it. This is the episode §6.1 cites for the correctness axis, and no reported number comes from the pre-repair bank. **(ii) Evaluated revision.** The pool every reported run retrieved over, shipped as `data/training/crafting_examples.json`. **(iii) Corrected revision (2026-07-22).** Shipped beside it as `crafting_examples.CORRECTED-2026-07-22.json`. 18 of the 25 crafting examples differ: the `stick` constant raised from 4 to 6 or 8 (10 examples), `oak_planks` from 8 to 12 with `oak_log` from 2 to 3 (9), an explicit `crafting_table` craft step added (7), a stone quantity raised (2), the `crafting/gold_shovel` prerequisite chain prepended (3→7 steps), a furnace step added to `crafting/stone_slab`, and three precondition edits including the one `planks`→`oak_planks` normalisation. The quantity edits are the ones Appendix U analyses; the two ingredient-set edits are the ones Appendix M uses. **(iv) Not repaired.** The 25 invalid commands of Appendix V are disclosed and left in place, since editing them would decouple the shipped bank from the runs.

Two limits of this record. The collecting, combat and build banks ship in a single revision each, so for the five non-crafting repairs of the audit above the pre-audit bytes are not recoverable from the release. And the seed files archive each plan and the number of demonstrations used, but not their identities, so the prompts cannot be re-derived from the release for those three categories. Neither limit touches a reported number: every SPR, PVR and McNemar value in this paper is a re-score of the *stored plans* (§4.3), which are archived verbatim regardless of which bank produced them.

F Denominator Sensitivity

One benchmark task, `craft/iron_horse_armor` (L3), has no crafting recipe in vanilla Minecraft 1.20.4 (`mcData` lists zero recipes for this item; it is loot/trade-only) and no loot or trade command exists in the bot's action space, so it is structurally infeasible under our protocol regardless of model or retrieval condition. We verified this directly rather than by assumption: re-running the validator over the two canonical result directories (102 occurrences of this task across all models, conditions, and seeds) confirms `strict_pass` and `pvr` are `false` in *all* 102 — a fully concordant failure. Widening the scan to every seed file in the repository, including superseded runs, yields 225 occurrences and likewise zero passes under either metric. Because a task that fails identically in both arms of every paired comparison contributes to neither McNemar's b nor c, its inclusion cannot affect any paired significance test; it only depresses every condition's absolute pass rate by a constant one task.

Table 5 reports Table 1 and Table 16 SPR values recomputed with this task excluded from the denominator (n=59). The numerator is identical in both columns for every row — the task never counts as a pass under either denominator, so this changes only the percentage, not the pass count. Every McNemar $b/c/p$ in the main text and in Table 11 below is unchanged; reported SPR percentages shift by at most +1–2 pp and no significance conclusion changes in either direction. We report the pre-specified /60 result as primary throughout the paper and treat this table as a sensitivity check confirming that result is not an artefact of the one infeasible task.

G Statistical Significance

The per-model McNemar exact tests (Appendix O; b/c = SAER-only / no_ret-only passes) ground the Main Results claims: SAER's per-model effect is Bonferroni-significant (α'=0.0038) for four of the five primary-study models, and nominally significant for the remaining one (Qwen3-8B,

Table 5: SPR sensitivity: primary /60 result vs. /59 result excluding the structurally invalid `craft/iron_horse_armor` task.

Model	Cond.	SPR /60	SPR /59
Qwen3-8B	no_ret	14/60 (23%)	14/59 (24%)
Qwen3-8B	saer_cold	26/60 (43%)	26/59 (44%)
GPT-4o	no_ret	23/60 (38%)	23/59 (39%)
GPT-4o	saer_cold	41/60 (68%)	41/59 (69%)
Claude S.4.5	no_ret	25/60 (42%)	25/59 (42%)
Claude S.4.5	saer_cold	51/60 (85%)	51/59 (86%)
Qwen2.5-Max	no_ret	19/60 (32%)	19/59 (32%)
Qwen2.5-Max	random	38/60 (63%)	38/59 (64%)
Qwen2.5-Max	bm25	42/60 (70%)	42/59 (71%)
Qwen2.5-Max	saer_cold	49/60 (82%)	49/59 (83%)
DeepSeek-V3	no_ret	33/60 (55%)	33/59 (56%)
DeepSeek-V3	saer_cold	52/60 (87%)	52/59 (88%)
Gemma4-26B	no_ret	44/60 (73%)	44/59 (75%)
Gemma4-26B	saer_cold	48/60 (80%)	48/59 (81%)
Gemma4-31B	no_ret	46/60 (77%)	46/59 (78%)
Gemma4-31B	saer_cold	53/60 (88%)	53/59 (90%)

p=0.0075). Under the post hoc Holm step-down over the same 13-test family (Appendix O) all five per-model effects are family-wise significant; Bonferroni remains the pre-specified primary analysis. All five primary-study paired-difference 95% confidence intervals (method for matched proportions of Newcombe [1998]) exclude zero, from $[+7, +32]$ pp (Qwen3-8B) to $[+36, +61]$ pp (Qwen2.5-Max). The Gemma4 replication panel is reported under both SAER arms. Under warm-start (`saer_full`, generated before the recipe-resolver fix) the arms read 48/60 SPR with 57/60 PVR for Gemma4-26B and 51/60 with 58/60 for Gemma4-31B, against the same `no_retrieval` baselines, giving b=5, c=1, p=0.219 and b=5, c=0, p=0.0625; under the primary study's cold-start protocol — same `saer_cold` batch and the same validator, though the panel's `no_retrieval` arm was generated 6–8 days after the primary study's (Appendix N) — they are +7 pp (b=8, c=4, p=0.388) for Gemma4-26B and +11 pp (b=7, c=0, p=0.0156) for Gemma4-31B. All four lie outside the 13-test family and are uncorrected; the panel is excluded from the family because it lies outside the pre-specified grid, not because its cold-start arm is less comparable. Under the debiting scorer both cold-start gains vanish (Gemma4-26B 35→28, p=0.118; Gemma4-31B 37→37; Table 23). The more informative fact about this panel is that both models already score above the L2 wall without retrieval, leaving little for retrieval to recover. The 13-test family (composition in §4.3) is the study's fixed evaluation grid, set before results were examined and reported in full regardless of outcome: no comparison was added or dropped based on significance. The Holm analysis re-thresholds that same fixed grid — it adds no test and removes none — but the decision to run it was made after the results were seen, so it is labelled post hoc throughout.

The ablation and cross-model panels (Appendix O) add that all retrieval conditions — including random — significantly beat no-retrieval for Qwen2.5-Max (all $p<0.001$; SAER-vs-BM25 there examined in §5.2), Qwen2.5-Max and DeepSeek-V3 with SAER significantly outperform Qwen3-8B with SAER (the only two cross-model tests against it), and DeepSeek-V3 and Qwen2.5-Max converge (b=5, c=2, p=0.453, n.s.).

H Live Execution: a Framework-Bound Axis

To test whether static plan quality transfers to embodied execution, we ran 21 of the 60 benchmark tasks — seven per tier, chosen for staging tractability rather than by a pre-specified rule — on a live Minecraft 1.20.4 server under DeepSeek-V3 in two conditions (no_retrieval, saer_cold) × three seeds (126 trials; Appendix S), scoring *strict* completion (goal item obtained) and *prerequisite-chain* completion (direct ingredients assembled, terminal craft excluded).

Live does not separate the conditions, and the reason is diagnostic. The 21 tasks split seven per tier, so a per-tier denominator of 21 counts trials (7 tasks × 3 seeds) while *21 tasks* counts tasks. At the L2 discrimination tier both conditions floor at 2/21 trials (10%) under *both* metrics, and 11

of 21 tasks fail under both in all three seeds — a symmetry no knowledge-driven account predicts. A failure-mode audit places most failures outside the knowledge axis: 33/42 (79%) of no_retrieval failures are framework-execution errors (Mineflayer sync races) against 2 prerequisite-chain gaps, and 21/46 saer_cold failures trace to a since-fixed defect mis-marking bundled craft steps as successful (Appendix S); since the prerequisite-chain metric excludes the terminal craft yet the L2 floor holds, the failure sits *upstream*. At L1 the two conditions are close (strict 15/21 vs. 18/21, tying at 18/21 on the prerequisite-chain metric), and SAER's examples lengthen plans (5.4 vs. 4.3 steps). Live completion is therefore governed by an execution-reliability axis floored equally for both conditions, leaving no headroom for a plan-quality difference: a scope limitation, not evidence either way (details in Appendix S).

I Combat Subgroup: An Existence Probe

The combat probe (design §4.4; per-condition table in Appendix S) reproduces the existence effect on the combat subgroup: `no_retrieval` passes 0/5 while random, BM25 and `saer_cold` each pass the same 3/5 tasks. The split sits at L1–L2 (0/1 and 0/2 without retrieval vs. 1/1 and 2/2 with it); on the runtime verdicts quoted here both L3 tasks fail under every condition, so the top tier has no headroom. (Under the uniform re-scoring of §4.3, `combat/kill_blaze` passes under `saer_cold`, which reads 4/5 rather than 3/5; both sets are given in Appendix S.) It cannot rank the variants: the margin is at most one task, and at $n{=}5$ even a perfect 5–0 discordant split gives $p{=}0.0625$ two-sided. These five tasks are members of the 60-task benchmark and are already counted in every $/60$ cell of Table 1; the probe re-runs them with the near-duplicate templates removed, so it is a subgroup analysis of the primary study and not an independent replication of it. We therefore read combat as an underpowered existence check only and draw the relevance conclusion solely from the 60-task ablation (§5.2).

J Weight Configuration Ablation Detail (Exploratory, 35-Task Benchmark)

Exploratory; not confirmatory for the deployed scorer. This study used the original 35-task benchmark, Qwen-8B (`qwen3-vl-8b-instruct`, a different model from the main study's Qwen3-8B), and an older *four*-component formulation with 4:3:2:1 weights, in which $\alpha, \beta, \gamma, \delta$ corresponded to ingredient overlap, keyword overlap, mining-tool compatibility and difficulty match. The frozen 60-task results are produced by the current cold-start scorer, which is *three*-component — $(\alpha, \beta, \delta){=}(0.50, 0.35, 0.15)$ with the mining-tier term at $\gamma{=}0$ under an empty inventory (§3.3). The configuration exercised here is therefore not the configuration deployed in Table 1. The historical result motivates a new ablation of the deployed weights; it does not validate them, and no main-text claim rests on it. Table 6 reports the configurations tried.

The template-correctness episode, on the same benchmark. §6.1 cites one contrast for the correctness facet, and it comes from this benchmark rather than the frozen one. Six L2 crafting demonstrations (`iron_axe`, `iron_pickaxe`, `iron_shovel`, `iron_hoe`, `shears`, `shield`) declared `stone_pickaxe:1` as a step-2 precondition while no earlier step crafted a wooden or stone pickaxe, and the planners followed the template verbatim. Under the scorer of the time, `saer_cold` read **1/13** at L2 against `random_retrieval`'s **6/13** — the retriever that selects the most structurally similar demonstration is the one that inherits its defect, so a relevance-ranked retriever can fall *below* a random one when the bank is wrong. That is the whole of what this episode licenses, and three limits bound it. It is a single run per arm with no paired test, so the figures carry no interval. After the repair `saer_cold` read 11–12/13 across three runs, but two validator bugs were fixed in the same batch (a `None (assume access:trees)` string parsed as an item requirement, and `produces` inference running only when a step declared no outputs), so the post-repair half cannot be attributed to the bank. And the repair predates the freeze of the 60-task benchmark and every run reported in this paper: the earliest timestamp in Table 10 is later, so no reported number comes from the defective bank. A controlled version of this contrast — the same frozen benchmark, defective and repaired banks crossed with SAER and random — is the experiment the correctness facet still lacks.

Within the iron-tool category, four of the six iron training examples share an identical declared ingredient set (`shears` lacks the stick, `shield` substitutes planks); ingredient overlap alone cannot

Table 6: Weight configuration ablation: averaged strict SPR over 3 seeds (original 35-task benchmark, Qwen-8B, empty starting inventory; the older four-component formulation, not the cold-start vector of §3.3).

Configuration	α	β	γ	δ	Avg. SPR
ingredient_only	1.00	0.00	0.00	0.00	28.6%
keyword_only	0.00	1.00	0.00	0.00	36.2%
equal (0.25×4)	0.25	0.25	0.25	0.25	39.0%
SAER (4:3:2:1)	**0.40**	**0.30**	**0.20**	**0.10**	**73.3%**

differentiate them, so retrieval degrades to random selection within the correct material tier. The 4:3:2:1 assignment was fixed before benchmark evaluation.

What this table does and does not corroborate. A weight vector invites the reading that its *ordering* $\alpha{>}\beta{>}\gamma{>}\delta$ was established somewhere. It was not, and this table is the closest thing to a test of it, so we state precisely what it supports.

Not corroborated: the ordering. The two single-dimension rows run the other way — keyword alone (36.2%) scores above ingredient alone (28.6%), while the ordering places ingredient first. Single dimensions in isolation are a weak probe of a dimension's marginal contribution inside a combination, so we do not read the reversal as refuting the ordering either; the honest statement is that this table is silent on the ordering, and no experiment in this paper measures it. The weight ordering of §3.3 should therefore be read as the design rationale it is, fixed before evaluation, not as an empirically ranked result.

Not corroborated: the necessity of calibration. The $+34$ pp gap over equal weighting reads at first as confirming that a calibrated rather than uniform combination is necessary. It does not support that reading, and we set it aside rather than carry it forward. Appendix K re-runs the comparison against the *deployed* weights on the frozen 60-task benchmark, under the fixed validator, on two models, and finds no detectable difference between calibrated and uniform weighting — with the direction of the (non-significant) difference inconsistent across the two models.

What does survive. Combining the dimensions beats both single-dimension rows in this table (73.3% against 28.6% and 36.2%). On the frozen benchmark we show only that the combination is not interchangeable with any single dimension — collapsing to one re-orders the selected demonstrations for 48–58 of 60 tasks — not that it scores higher there. The deployed weights are best described as one pre-specified working point among several that perform comparably, and the 35-task gap above is retained as a record of what the older four-component configuration did on the older benchmark, not as evidence about the deployed scorer.

K Deployed-Configuration Ablations (Exploratory)

Exploratory; outside the confirmatory family. The comparisons below are run on the frozen 60-task benchmark under the fixed validator of §4.3, but they were specified after the primary study and are excluded from the 13-test Bonferroni family of §4.3. They are reported to address the question Appendix J raises — whether the *deployed* weights are better than uniform ones — and they find no advantage for them. They cannot establish equivalence either: restricting each weight comparison to its divergent subset leaves $b{+}c \leq 2$ discordant tasks, and the smallest two-sided exact p attainable at $b{+}c$ discordances is $2 \cdot 0.5^{\,b+c}$, i.e. 0.5 at $b{+}c{=}2$ and 1.0 at $b{+}c \leq 1$. The four weight rows of Table 7 therefore could not have reached significance under any outcome; read them as inconclusive, not as demonstrated equivalence. The k comparisons, run on all 60 tasks, have more discordance budget ($b{+}c{=}8$ and 4) but the $k{=}5$ row is likewise pinned at its attainable floor of 0.125.

Two weight comparisons are run on the subset of tasks for which the two configurations' ordered top-k retrieval actually diverges, since where retrieval is identical the prompts are identical, so any outcome difference there cannot be a weight effect; each comparison therefore has its own 18-task subset and its own baseline count. The top-k comparisons run on all 60 tasks. Every right-hand arm was generated on 07-31, 40 days after the 06-21 `saer_cold` arm it is compared with, and both models are served under provider aliases (`qwen-max`, `deepseek-chat`), so an endpoint change is a

rival explanation for any difference here, as it is for the relevance test (Appendix N). Table 7 reports all six. As elsewhere, b counts tasks passing only under the right-hand configuration and c only under the left-hand one, so left $+ b - c =$ right.

Table 7: Deployed-configuration ablations (majority vote over 3 seeds, McNemar exact). "Deployed" is the cold-start $(\alpha, \beta, \delta)=(0.50, 0.35, 0.15)$ scorer of §3.3 at $k=3$; "equal" weights the same active dimensions uniformly; "5-dim" adds a prerequisite-chain term to the four terms of Equation 1. Source runs in Table 10. No comparison reaches nominal significance, and the sign of the weight effect is inconsistent across the two models.

Comparison	Model	n	Left → Right	b	c	p
deployed → equal	Qwen2.5-Max	18	13 → 15	2	0	0.500
deployed → equal	DeepSeek-V3	18	15 → 14	0	1	1.000
deployed → 5-dim	Qwen2.5-Max	18	16 → 15	0	1	1.000
deployed → 5-dim	DeepSeek-V3	18	16 → 16	0	0	1.000
$k=3 \rightarrow k=1$	Qwen2.5-Max	60	49 → 43	1	7	0.0703
$k=3 \rightarrow k=5$	Qwen2.5-Max	60	49 → 45	0	4	0.125

What this shows. Calibrated and uniform weighting are indistinguishable on both models, and the direction of the (non-significant) difference reverses between them ($+2$ tasks on Qwen2.5-Max, -1 on DeepSeek-V3). Adding a fifth prerequisite-chain dimension does not help either. We therefore withdraw the reading that Appendix J's $+34$ pp gap over equal weighting establishes that calibration is necessary; the deployed weights are one defensible working point, not a demonstrated optimum. The top-k rows are similarly indicative rather than confirmatory: $k=3$ is the best of the three values tried on the one model tested, with an inverted-U shape, but neither contrast reaches nominal significance. None of this affects the primary claims, which compare retrieval against no retrieval rather than one retrieval configuration against another.

L Three-Way Retrieval Comparison (Exploratory, 42-Task Benchmark)

Exploratory; predates the final benchmark and the validator fix. This study uses the earlier 42-task benchmark and is not combined with any confirmatory result.

Table 8 supplements §2 with a SBERT comparison on the original 42-task benchmark with Qwen-8B (`qwen3-vl-8b-instruct`, as in Appendix J, not the main study's Qwen3-8B). SBERT achieves the same overall pass rate as BM25 (74%) but with a complementary per-level profile: stronger at L3 (79% vs. 50%) and weaker at L1/L2 (67–75% vs. 83–88%). Neither method dominates overall ($p=1.000$, n.s.). These runs predate the validator fix of §4.3 and were not re-validated (SBERT embeddings are no longer reproducible at the same model revision), so this table should be read as the earlier 42-task study, not as part of the 60-task results; no paper claim rests on them.

M Structural-Overlap Robustness Check

Exact target-name disjointness does not preclude structural (ingredient/prerequisite) overlap, so we additionally computed, for each of the 44 CRAFT-category held-out tasks, the ingredient-set

Table 8: Supplementary: BM25 vs. SBERT vs. SAER-cold (Qwen-8B, 42-task benchmark, majority vote).

	BM25	SBERT	SAER-cold
Overall (mv)	31/42 (74%)	31/42 (74%)	**34/42 (81%)**
L1 (12)	10/12 (83%)	8/12 (67%)	**11/12 (92%)**
L2 (16)	14/16 (88%)	12/16 (75%)	**15/16 (94%)**
L3 (14)	7/14 (50%)	**11/14 (79%)**	8/14 (57%)
Mean seed rate	73.8%	61.9%	**74.6%**

Jaccard similarity to its nearest demonstration (query set = the task's declared final-craft materials; demonstration set = the union of `items.inventory` preconditions across all of its plan steps, i.e. its full prerequisite chain). The nearest demonstration is never a zero-overlap match (min=0.143, mean=0.386, max=1.000, n=44); the 16 COLLECT/COMBAT tasks are excluded from this specific metric because they have no declared final-craft materials by task design (they collect raw blocks or kill mobs rather than craft from ingredients), not because they are structurally unrelated to the demonstration pool.

This measure is asymmetric — the query side uses only final-craft materials while the demonstration side uses each example's full prerequisite chain — which could overstate similarity. As a robustness check we recomputed a symmetric variant, expanding the query side to its own full prerequisite chain via the same static dependency resolution used elsewhere in the pipeline (applied offline from an empty inventory). The symmetric measure is lower and flatter (min=0.000, mean=0.290, max=0.500, n=44) than the asymmetric one, with no task reaching the near-duplicate range (0/44 above 0.75). At stricter thresholds, the asymmetric measure has 14/44 tasks at or above 0.5, 4/44 at or above 0.75, and 4/44 at or above 0.9 (i.e. 4 near-duplicate matches), whereas the symmetric measure has only 3/44 at or above 0.5 and 0/44 at or above 0.75 or 0.9 — confirming that the apparent near-duplicates are an artefact of the asymmetric comparison. The four asymmetric-measure exact matches (Jaccard=1.0: `wooden_sword`, `oak_sign`, `wooden_shovel`, `oak_fence`, each matched to a wooden-tool demonstration) arise because these tasks' declared material sets contain only two items (a stick and a plank), so any other two-item wooden-tool set is trivially identical — not because of retrieval-pool duplication; none of the four survive under the symmetric measure.

Which bank revision. The crafting bank was revised after the evaluation runs; 18 of its 25 examples differ between the two revisions, in the edit classes enumerated in Appendix E. Both revisions ship in the code and data release, and all three overlap scripts (`jaccard_overlap_holdout_vs_demos.mjs`, `jaccard_overlap_symmetric.mjs`, `overlap_tercile_stratify.mjs`) take the bank path from the `CRAFTING_BANK` environment variable, so either block of Table 9 is reproducible. The two questions this section answers call for different revisions. For the *contamination bound*, the corrected revision is the conservative choice, and the figures in the two paragraphs above are computed on it: the two edits that touch ingredient sets — repairing the `crafting/gold_shovel` prerequisite chain (3→7 steps) and normalising one `planks`→`oak_planks` precondition string — both enlarge the demonstration side and therefore *raise* every measured overlap; the remaining edits change quantities only, which this metric does not read. Under the evaluation-time revision — the pool the reported runs actually retrieved over — overlap is uniformly lower: asymmetric min=0.167, mean=0.317; symmetric mean=0.188, max=0.333, with **0/44** tasks at or above 0.5 (versus 3/44), and **one** exact asymmetric match rather than four. We use the higher estimates for the bound so that it is not flattered by the pool we ship. For *where the gain occurs*, the evaluation-time revision is the relevant one, since it is the pool the runs actually retrieved over; §4.1 quotes it, and it leads Table 9.

We further stratified SAER's SPR gain on Qwen2.5-Max (`no_retrieval`→`saer_cold`) by overlap tercile (low/mid/high, n=15/15/14) to test whether the gain is concentrated in high-overlap, near-duplicate tasks — which would undercut a generalisation claim. On the evaluation-time bank the gain rises monotonically with overlap ($+6/+8/+10$ asymmetric, $+5/+9/+10$ symmetric; Table 9, upper block). This does not support a near-duplicate explanation, for two reasons. First, under that bank there are no near-duplicates for the gain to concentrate in: the symmetric measure spans 0.000–0.333 across all 44 tasks, so its "high" tercile (0.231–0.333) is a low-overlap stratum in absolute terms, and a gradient across a range that never reaches 0.5 cannot be retrieval of a near-copy. Second, the tercile is confounded with difficulty by construction: that stratum is seven `iron_*` items, three `stone_*` items, `cauldron`, `minecart`, `compass` and `lever` — the iron- and stone-tier multi-hop chains, which share ingredients with the demonstrations and with each other simply because those tiers have few distinct inputs. Its no-retrieval floor is **0/14**, the L2 bottleneck this paper is about, whereas the corresponding low-overlap tercile starts from 7/15. Under the corrected bank the direction reverses: the gain is largest in the low- and mid-overlap terciles (lower block), so the gradient is not stable across bank revisions, and we draw no direction from either panel. Overlap tercile is therefore substantially a proxy for prerequisite depth; what flips between the two panels is which end of that depth axis the terciles land on, not whether the gain comes from retrieving near-copies.

Gathering and combat tasks. Ingredient Jaccard is empty for the 16 non-crafting tasks, which declare no required materials, so we check them for the overlap that does apply to them: whether

Table 9: SAER's SPR gain on Qwen2.5-Max (`no_retrieval`→`saer_cold`), stratified by structural-overlap tercile, under both bank revisions. Jaccard ranges are given per cell. Reproduce the upper block with the shipped default and the lower block with `CRAFTING_BANK=data/training/crafting_examples.CORRECTED-2026-07-22.json`.

Overlap tercile (n)	Low (15)	Mid (15)	High (14)
Evaluation-time bank (the pool the runs retrieved over; primary)			
Asymmetric	6→12	1→9	3→13
Jaccard range	.167–.250	.250–.333	.333–1.00
Symmetric	7→12	3→12	0→10
Jaccard range	.000–.182	.182–.214	.231–.333
Corrected bank (conservative contamination bound)			
Asymmetric	0→10	0→12	10→12
Jaccard range	.143–.250	.250–.400	.500–1.00
Symmetric	6→12	0→13	4→9
Jaccard range	.000–.273	.273–.300	.300–.500

a task's goal item is already the target of a demonstration in the bank it retrieves from (retrieval is category-filtered, so each category is compared with its own bank). The check (`scripts/eval/collect_overlap_check.mjs`) runs first on the combat tasks as a positive control and aborts unless it recovers the five known near-duplicates of §4.4; it does (5/5). On the 11 gathering tasks against the 21 collecting demonstrations, no task's goal item is the final product of any demonstration (0/11); one, `gravel`, appears as an intermediate collection step in two demonstrations (`collect/flint`, `collect/blaze_rod`). The gathering tier therefore holds no near-duplicate of the combat kind.

N Number Provenance

Every SPR, PVR and McNemar value in the main text is recomputed from the per-seed plan files listed in Table 10 by a single script (`scripts/eval/mcnemar_table.mjs --rescore`), which re-validates all three stored seed plans of each run with the fixed validator (§4.3, *Re-validation*) and then applies the per-task majority vote. The script also asserts the McNemar identity $\mathrm{SPR}_{\text{saer}} = \mathrm{SPR}_{\text{no_ret}} + b - c$ for every model row, so the main text's cross-architecture table and Table 11 below cannot drift apart. Note that the per-run summary files shipped alongside the plan files were written at generation time, i.e. before the validator fix, and therefore do *not* match the reported numbers; the per-seed plan files are the authoritative record.

Generation epochs, and which comparison is exposed. Re-validation fixes scoring, not generation, so the timestamps below still matter. The existence step is the best-protected comparison in the paper: Qwen2.5-Max's `no_retrieval` and `random` runs are 2.5 hours apart in the same directory (20260604_213651 and 20260605_000159), and all five `no_retrieval` arms predate their `saer_cold` arms because the recipe-resolver fix forced the SAER side to be regenerated (§4.3). The relevance step is the worst-protected one. Four models' BM25 arms were generated inside the same 06-20/06-21 window as their `saer_cold` arms, but Qwen2.5-Max's BM25 run is from 06-05 and its `saer_cold` run from 06-21 — 16 days, across a directory change, on `qwen-max`, which is a provider alias rather than a date-pinned identifier (§6.2). That pair is the family's only relevance test (b=7, c=0), so an endpoint change during those 16 days is a rival explanation we cannot exclude, and re-validating stored plans does not touch it. Settling it needs a same-window rerun of that one arm, which we have not done.

O McNemar Tests: Per-Model, Ablation and Cross-Model Panels

Table 11 reports every test in the pre-specified Bonferroni family (Appendix G): five per-model `saer_cold` vs. `no_retrieval` comparisons, the five-row Qwen2.5-Max ablation panel, and three cross-model SAER comparisons, k=13, so α'=0.05/13=0.0038. One test appears in two panels (Qwen2.5-Max `saer_cold` vs. `no_retrieval` is both the per-model test and the third ablation row); we count it twice, which makes the correction conservative rather than lenient, and no verdict changes

Table 10: Source runs. Cells give the run timestamp; the directory prefix is `scripts/eval/results/` plus ^{c} = `canonical/`, ^{f} = `fixed_mcdata/`, or no marker for the results root. Each run contributes three seed files (`_s1`–`_s3`).

Model	Condition	Run timestamp
Qwen3-8B	no_retrieval	20260605_213048^{c}
	bm25	20260620_191130^{c}
	saer_cold	20260621_004648^{f}
GPT-4o	no_retrieval	20260605_213059^{c}
	bm25	20260621_121743^{f}
	saer_cold	20260621_013640^{f}
Claude S. 4.5	no_retrieval	20260605_233949^{c}
	bm25	20260621_144209^{f}
	saer_cold	20260621_021308^{f}
Qwen2.5-Max	no_retrieval	20260604_213651^{c}
	random	20260605_000159^{c}
	bm25	20260605_004805^{c}
	saer_cold	20260621_001120^{f}
DeepSeek-V3	no_retrieval	20260604_231018^{c}
	bm25	20260621_115631^{f}
	saer_cold	20260621_011658^{f}
Gemma4-26B	no_retrieval	20260611_204737
	bm25	20260621_153049^{f}
	saer_cold	20260621_144219^{f}
	saer_full	20260612_025602
Gemma4-31B	no_retrieval	20260612_235009
	bm25	20260621_211600^{f}
	saer_cold	20260621_202849^{f}
	saer_full	20260613_122627
Deployed-configuration ablations (Appendix K); 18-task divergent subsets		
Qwen2.5-Max	equal weights	20260731_132239
	5-dim	20260731_133856
DeepSeek-V3	equal weights	20260731_135442
	5-dim	20260731_140056
Top-k ablation (Appendix K); full 60 tasks		
Qwen2.5-Max	k=1	20260731_142747
	k=5	20260731_151943
Combat probe (Qwen2.5-Max, 20260703): `combat_holdout/` (R3), `combat_holdout_r4/` (R4)		

at k=12 (α'=0.0042). The two-model Gemma4 replication panel is excluded from this family (Appendix G). The same 13 hypotheses are re-examined under Holm's step-down procedure below; the family membership is identical, only the thresholding differs. The duplicate is conservative for Bonferroni but exactly neutral for Holm: the duplicated p is the smallest in the family, so removing it leaves every remaining threshold $\alpha/(m-i+1)$ unchanged and the set of rejected hypotheses is the same at k=12.

The last four rows quantify how closely the replication panel tracks BM25. With `saer_full` each model has $b+c$=2 discordant tasks of 60, i.e. 58/60 (97%) concordant; with the cold-start arm Gemma4-26B has 8 (52/60, 87%) and Gemma4-31B none (60/60). They are listed for provenance only. The panel is excluded from the Bonferroni family for the reason given in Appendix G — it is a replication panel outside the pre-specified evaluation grid — so these are nominal p-values attached to a descriptive comparison. Its cold-start arm is protocol-comparable to the primary study and is reported in Table 1 and Appendix G; the exclusion does not turn on comparability.

Per-model `saer_cold` vs. `bm25`. §5.2 summarises the five models' SAER-over-BM25 margins (four higher, GPT-4o one task lower); Table 12 tabulates all five so they can be checked directly. **Only the Qwen2.5-Max row belongs to the pre-specified 13-test family** (it is the fifth ablation row); the other four are descriptive and were not added to the family, so the Bonferroni and Holm counts of Appendix G are unchanged by this table. The family-level statement about relevance is the one-sided sign test over the 4–1 split (p=0.188), not any single row here.

Holm step-down: a post hoc sensitivity analysis, not a replacement. The Bonferroni correction above is the pre-specified primary analysis, and every headline verdict in the main text is a Bonferroni verdict. Bonferroni is, however, conservative: it holds all 13 tests to the same threshold no matter

Table 11: McNemar exact test (majority vote, n=60). b/c = condition-A-only / condition-B-only passes. $\star$=p<0.05, $\star\star$=p<0.01, $\star\star\star$=p<0.001, n.s. = $p \geq 0.05$; stars mark nominal p, and only $p<\alpha'=0.0038$ survives the pre-specified Bonferroni correction. Holm verdicts over the same 13 tests are in Table 13.

Comparison	b	c	$b+c$	p-value
Per-model: `saer_cold` vs. `no_retrieval`				
Qwen3-8B	15	3	18	0.0075 $\star\star$
GPT-4o	19	1	20	<0.001 $\star\star\star$
Claude S. 4.5	26	0	26	<0.001 $\star\star\star$
Qwen2.5-Max	30	0	30	<0.001 $\star\star\star$
DeepSeek-V3	21	2	23	<0.001 $\star\star\star$
Qwen2.5-Max four-condition ablation				
random vs. no_ret	19	0	19	<0.001 $\star\star\star$
bm25 vs. no_ret	24	1	25	<0.001 $\star\star\star$
saer_cold vs. no_ret	30	0	30	<0.001 $\star\star\star$
bm25 vs. random	9	5	14	0.424 n.s.
saer_cold vs. bm25	7	0	7	0.0156 $\star$
Cross-model SAER comparisons				
Qwen2.5-Max vs. Qwen3-8B	23	0	23	<0.001 $\star\star\star$
DeepSeek-V3 vs. Qwen3-8B	26	0	26	<0.001 $\star\star\star$
DeepSeek-V3 vs. Qwen2.5-Max	5	2	7	0.453 n.s.
Outside the family: replication panel, `saer_full` vs. `bm25`				
Gemma4-26B	1	1	2	1.000 n.s.
Gemma4-31B	0	2	2	0.500 n.s.
Outside the family: replication panel, `saer_cold` vs. `bm25`				
Gemma4-26B	4	4	8	1.000 n.s.
Gemma4-31B	0	0	0	1.000 n.s.

Table 12: Per-model `saer_cold` vs. `bm25` (majority vote, n=60). b/c = SAER-only / BM25-only passes. Descriptive: only the Qwen2.5-Max row is a member of the pre-specified 13-test family; the other four rows carry uncorrected nominal p-values and no family-wise verdict. Reproduce with `node scripts/eval/mcnemar_table.mjs --rescore` (rows labelled `bm25 delta:`).

Model	`bm25`	`saer_cold`	b	c	p-value
Qwen3-8B	21/60	26/60	6	1	0.125 n.s.
GPT-4o	42/60	41/60	6	7	1.000 n.s.
Claude S. 4.5	44/60	51/60	8	1	0.0391 $\star$
Qwen2.5-Max‡	42/60	49/60	7	0	0.0156 $\star$
DeepSeek-V3	46/60	52/60	7	1	0.0703 n.s.

‡ in the pre-specified family (Table 11, ablation panel).

Table 13: Holm step-down over the same 13-test family, α=0.05, m=13. Tests are sorted by p; rejection requires $p \leq \alpha/(m-i+1)$ and stops at the first failure. The nine $p<0.001$ entries are shown at their exact binomial values so the comparisons can be checked directly. B/H = rejected under Bonferroni / Holm.

i	Test	p	$\alpha/(m-i+1)$	B / H
1	per-model Qwen2.5-Max	1.9×10^{-9}	0.00385	✓ / ✓
2	ablation saer_cold vs. no_ret	1.9×10^{-9}	0.00417	✓ / ✓
3	per-model Claude S. 4.5	3.0×10^{-8}	0.00455	✓ / ✓
4	cross DeepSeek vs. Qwen3-8B	3.0×10^{-8}	0.00500	✓ / ✓
5	cross Qwen2.5-Max vs. Qwen3-8B	2.4×10^{-7}	0.00556	✓ / ✓
6	ablation bm25 vs. no_ret	1.5×10^{-6}	0.00625	✓ / ✓
7	ablation random vs. no_ret	3.8×10^{-6}	0.00714	✓ / ✓
8	per-model GPT-4o	4.0×10^{-5}	0.00833	✓ / ✓
9	per-model DeepSeek-V3	6.6×10^{-5}	0.01000	✓ / ✓
10	per-model Qwen3-8B	0.0075	0.01250	— / ✓
11	ablation saer_cold vs. bm25	0.0156	0.01667	— / ✓
12	ablation bm25 vs. random	0.424	0.02500	— / —
13	cross DeepSeek vs. Qwen2.5-Max	0.453	0.05000	— / —

how small the smallest p-values are. We therefore also report Holm's sequentially rejective procedure [Holm, 1979] over the identical 13 hypotheses. Holm controls the same family-wise error rate in the strong sense and is uniformly more powerful, so relative to Bonferroni it can only add rejections, never withdraw one. We state plainly that this analysis was added *after* the results were known. It is reported alongside the pre-specified correction rather than in place of it, and it changes no p-value, no effect size and no experimental number in this paper — only which of the two thresholding rules a given p is being measured against. Choosing between them after the fact, and reporting only the more favourable one, would not be a defensible analysis; both are given wherever they disagree.

Holm sorts the 13 p-values ascending and compares the i-th against $\alpha/(m-i+1)$ with m=13, stopping at the first non-rejection and retaining every hypothesis from there on (Table 13). Nine tests clear even the strictest, rank-1 threshold $\alpha/13$=0.0038 and are rejected under both procedures. Holm additionally rejects two: the Qwen3-8B per-model test (p=0.0075 against $\alpha/4$=0.0125) and `saer_cold` vs. BM25 (p=0.0156 against $\alpha/3$=0.0167). The step-down then halts at rank 12 (p=0.424 against $\alpha/2$=0.025), so both null results remain null under either rule. Holm thus rejects 11 of 13 where Bonferroni rejects 9 of 13, and the two affected claims are flagged as such in §4.3, §5.1 and §5.2.

Robustness. The Holm verdict does not depend on the precise values of the nine smallest p-values, only on the fact that they are small. Every one of them falls below the rank-1 threshold 0.0038 — the largest is 6.6×10^{-5} (DeepSeek-V3 per-model, b=21, c=2), a margin of roughly 58× — so any permutation of their internal ordering still satisfies all nine of the ranks-1–9 comparisons and leaves ranks 10–13 untouched. The two Holm-only rejections are the only verdicts in the family that turn on a p-value close to its threshold (0.0075 vs. 0.0125, and 0.0156 vs. 0.0167; the latter by a factor of 1.07), and both are reported as Holm-only wherever they appear.

Sensitivity to the family size. What the two Holm-only rejections *do* depend on is m, and we report that dependence rather than leave it to be discovered. Holding the observed p-values fixed and enlarging the family: `saer_cold` vs. BM25 (p=0.0156, rank 11) is rejected at m=13 ($\alpha/3$=0.0167) and fails at m=14 ($\alpha/4$=0.0125); the Qwen3-8B per-model test (p=0.0075, rank 10) survives to m=15 and fails at m=16. The first of these has no margin at all: the relevance verdict holds only at the family size we pre-specified. Two enlargements are worth naming. Adding the four sibling per-model `saer_cold`-vs-BM25 tests of Table 12 — same hypothesis, four more models — gives m=17, at which Holm and Bonferroni both reject 9 and *both* Holm-only rejections disappear. Adding every test reported anywhere in this paper ($m\approx41$) still rejects the nine core comparisons under both rules. The existence result and the four per-model SAER effects are therefore robust to the family boundary; the relevance result is not, and §5.2 states it as such.

Where the Retrieval Signal Applies

The ingredient dimension $\alpha \cdot I$ is the only component that reads the recipe graph, and it is identically zero on the 16 gathering and combat tasks (§3.3; also on two CRAFT tasks, below). That gives a mechanism test the paper would otherwise lack: if SAER's edge over BM25 comes from ingredient structure, it should be absent exactly where that dimension is inert. Table 14 splits the paired counts accordingly. The subsets are restrictions of the same majority vote, not a re-analysis, and the script asserts that the two strata sum to the printed b and c.

Table 14: Paired counts by task category. CRAFT (n=44) is the category where $\alpha \cdot I$ is defined — 42 of the 44 actually score non-zero against some candidate — and the complement (n=16) is where it is identically zero by construction. Reproduce with `node scripts/eval/mcnemar_table.mjs --rescore --strata`.

	All 60		CRAFT (44)		Non-CRAFT (16)	
Model	b/c	p	b/c	p	b/c	p
existence: `saer_cold` vs. `no_retrieval`						
Qwen3-8B	15/3	0.0075	11/1	0.0063	4/2	0.688
GPT-4o	19/1	<0.001	15/0	<0.001	4/1	0.375
Claude S. 4.5	26/0	<0.001	21/0	<0.001	5/0	0.0625
Qwen2.5-Max	30/0	<0.001	24/0	<0.001	6/0	0.0313
DeepSeek-V3	21/2	<0.001	16/2	0.0013	5/0	0.0625
relevance: `saer_cold` vs. `bm25`						
Qwen3-8B	6/1	0.125	5/0	0.0625	1/1	1.000
GPT-4o	6/7	1.000	4/3	1.000	2/4	0.688
Claude S. 4.5	8/1	0.0391	8/1	0.0391	0/0	—
Qwen2.5-Max	7/0	0.0156	7/0	0.0156	0/0	—
DeepSeek-V3	7/1	0.0703	6/1	0.125	1/0	1.000

The split runs in the direction the mechanism predicts. *Existence* gains appear in both subsets — off CRAFT every model is positive ($b-c = 2, 3, 5, 6, 5$), so having any same-category demonstration helps even when no ingredient set exists to compare. The *relevance* increment does not: off CRAFT the five models read $(b, c) = (1, 1), (2, 4), (0, 0), (0, 0), (1, 0)$ — three ties, one negative, one single task — against $(5, 0), (4, 3), (8, 1), (7, 0), (6, 1)$ on CRAFT. All seven discordant tasks behind the family's one relevance test (Qwen2.5-Max, b=7, c=0) are CRAFT tasks, but only six lie among the 42 with a non-zero I: the seventh, `craft/flint_and_steel`, is ranked on K and L alone. We do not pool these counts into a single test: the five models are evaluated on the same 60 tasks, so their pairs are not independent.

Two consequences. First, the increment SAER adds over a lexical retriever is concentrated in the tasks whose ingredient structure it can read, which is what its dominant weight is for; the paper's relevance claim should be read as scoped to CRAFT tasks, with six of its seven discordant tasks among those 42. Second, 18 of the 60 tasks (16 non-CRAFT plus `craft/flint_and_steel`, whose three-word name the target-item parser cannot match because it drops "and" as a stop-word, and `craft/iron_horse_armor`, which has no 1.20.4 recipe) score SAER on K and L alone, so on 30% of the benchmark SAER *is* a keyword retriever with a three-valued difficulty prior — and on that 30% its record against BM25 is small and mixed: net $b-c$ of 0, -2, 0, $+1$ and $+1$ in the model order of Table 14, the Qwen2.5-Max $+1$ being `craft/flint_and_steel`.

Dropping the Contaminated Combat Subset

Five of the nine combat demonstrations target the same five mobs as the five held-out combat tasks and differ only by slug prefix, which the slug-level held-out check does not catch (§4.4). The probe of Appendix I excludes those demonstrations; the runs behind Table 1 do not, so every retrieval arm's $/60$ includes up to five tasks served by a near-duplicate. Table 15 recomputes the paired tests with those five tasks removed from the denominator.

No verdict in the 13-test family changes. The relevance test is numerically identical (b=7, c=0, p=0.0156) because none of its discordant tasks is a combat task; four of the five per-model existence

Table 15: Paired tests over the 55 non-combat tasks, against the reported 60-task values. Same majority vote, restricted denominator. Reproduce with `node scripts/eval/mcnemar_table.mjs --rescore --strata`.

Comparison	55-task	b/c	p	(60-task p)
existence: `saer_cold` vs. `no_retrieval`				
Qwen3-8B	13→23	12/2	0.0129	(0.0075)
GPT-4o	23→40	18/1	<0.001	(<0.001)
Claude S. 4.5	25→46	21/0	<0.001	(<0.001)
Qwen2.5-Max	18→44	26/0	<0.001	(<0.001)
DeepSeek-V3	33→48	17/2	<0.001	(<0.001)
ablation on Qwen2.5-Max				
`random` vs. `no_ret`	18→33	15/0	<0.001	(<0.001)
`bm25` vs. `no_ret`	18→37	20/1	<0.001	(<0.001)
`saer_cold` vs. `bm25`	37→44	7/0	0.0156	(0.0156)

Table 16: Qwen2.5-Max four-condition ablation (60 tasks, 3 seeds, majority vote). SPR and PVR are percentages of 60; *Gap*, L1, L2 and L3 are task counts (Gap = PVR−SPR in tasks, tiers out of 20), so the Gap column is not the difference of the two percentages beside it.

Condition	SPR	PVR	Gap	L1	L2	L3	Δ
no_ret	32%	47%	+9	17	2	0	—
random	63%	90%	+16	17	14	7	+31pp ⋆⋆⋆
bm25	70%	88%	+11	17	11	14	+38pp ⋆⋆⋆
saer_cold	82%	93%	+7	20	14	15	+50pp ⋆⋆⋆

tests stay below 10^{-3}, and Qwen3-8B moves from 0.0075 to 0.0129, nominal in both cases and clearing α' in neither. Absolute effect sizes shrink by 2.7–4.4 pp (Qwen2.5-Max existence +50.0 to +47.3 pp; `random` +31.7 to +27.3 pp; `bm25` +38.3 to +34.5 pp), so the headline percentages are the more generous of the two readings and should be quoted with that in mind. We keep the 60-task denominator as primary because the benchmark was frozen before any run and post hoc removal of tasks is exactly what §4.1 commits against; this table is the bound on what that choice costs.

Removing the contamination at its source — re-running generation with `--exclude-slugs` on the five near-duplicate demonstrations, as the probe does — would need new model calls and is not done here. The table above bounds the exposure from the task side instead, which is the part recoverable from the stored plans.

P Four-Condition Ablation: Full Panel

Table 16 gives the four-condition within-model ablation discussed in §5.2: knowledge *existence* (random retrieval +31 pp over no-retrieval, $p<0.001$) is the primary lever, and SAER exceeds BM25 on this model (saer_cold 82% vs. BM25 70%; b=7, c=0, p=0.0156) — nominally significant, but *not* family-wise significant under the pre-specified Bonferroni correction, since $0.0156>\alpha'=0.0038$, and family-wise significant only under the post hoc Holm step-down ($0.0156\leq\alpha/3=0.0167$, Appendix O). Per tier, BM25 trails random at L2 (11/20 vs. 14/20) but leads it at L3 (14/20 vs. 7/20); since BM25 and random do not differ overall (b=9, c=5, p=0.424), we read neither tier split as more than noise.

Q Thinking-Mode Ablation (Exploratory)

Table 17 reports the comparison.

To separate reasoning depth from template generalisation, we evaluate Qwen3-8B with extended thinking (`enable_thinking=true`; same 60-task benchmark and 3-seed majority-vote protocol). Its decoding may differ from the main study's Qwen3-8B runs: the release's thinking profile sets temperature 1 rather than 0.7, but that profile was committed after these runs, so the setting they used

Table 17: Qwen3-8B thinking-mode ablation (60 tasks, 3-seed majority vote). *Both* rows predate the evaluation fix and carry runtime verdicts, so they are reported for their qualitative pattern, not for point comparison against Table 1; re-scored figures are in the text.

Condition	SPR	L1	L2	L3	
no_ret	14/60 (23%)	14/20	0/20	0/20	—
saer_cold	20/60 (33%)	14/20	3/20	3/20	+10 pp (b=8, c=2, p=0.109, n.s.)

is not recorded. Thinking substantially raises L1 and leaves the SAER gain non-significant (+10 pp, b=8, c=2, p=0.109, n.s.). It does not lift L2: re-scored, the no-retrieval row reads 2/20 there against the non-thinking run's 3/20. L3 does move, 2/20 to 5/20, so "thinking does not unlock the multi-hop tiers" holds for L2 and is an overstatement for L3 on this single model. The L1 comparison must be read within one scoring regime: at runtime the non-thinking no-retrieval baseline is 8/20 and thinking is 14/20, while after re-scoring the same two runs read 9/20 (the value in Table 1) and 15/20. The lift is +6 tasks either way, but quoting the runtime 8/20 beside Table 1's re-scored 9/20 would be mixing the two. Thinking amplifies single-step reasoning but cannot substitute for the multi-hop procedural knowledge the model has not internalised — supporting our characterisation of the residual constraint as template-generalisation, not reasoning-depth (consistent with, and complementary to, the latent-depth ceiling of Xu et al. [2026]).

Caveat. *Both* rows predate the fix of §4.3. For the `saer_cold` row that reaches beyond scoring: it was *generated* with the recipe-resolver hook missing, so it retrieved with three of SAER's scoring dimensions disabled, and re-scoring repairs its verdicts, not its retrieval. Both rows are the verdicts recorded at runtime, and we report them only for the qualitative pattern, not for point comparison. The per-seed files are in the code and data release (`eval_20260606_024017_no_retrieval`, `eval_20260606_044848_saer_cold`), and re-scoring them with the current validator moves both conditions to 22/60, i.e. the `saer_cold` increment becomes 0 pp rather than +10 pp — still not significant, so the conclusion that thinking does not unlock the multi-hop tier is unchanged. The levelwise pattern is less stable than the totals: under re-scoring the no-retrieval row reads L1 15/20, L2 2/20, L3 5/20 rather than 14/0/0, so "L2/L3 stay at zero" is an artefact of the runtime scorer and should be read as "L2/L3 stay low" instead. This section is exploratory and no confirmatory claim rests on it.

R Capacity De-aliasing with Additional Sub-frontier Models (Exploratory)

Exploratory; provider-default decoding. The three models in this section are served through OpenRouter on a client path that does not forward the `temperature` parameter, so they run at provider defaults rather than the T=0.7 set for three of the five primary models (footnote below; per-model settings in Appendix D). They are reported as a family-diversity check on the capacity pattern, are not directly controlled against the main study, and are excluded from the confirmatory Bonferroni family of Appendix G.

§5.1 makes no causal capacity claim, but it does rest its smallest-model observation on a single data point, Qwen3-8B, which is confounded with the Qwen3 generation: a small gain there could reflect Qwen3-family template fragility rather than model scale. To separate the two, we run three additional *cross-family, non-Qwen* open-source models of comparable scale under the identical 60-task, 3-seed protocol with the `setRecipeResolver`-fixed static evaluator (§4.3): **Llama-3.1-8B-Instruct**, **Ministral-8B-2512**, and **Gemma-3-12B-it** (all served via OpenRouter).[1] These span ∼8–12B parameters and three independent families (Meta, Mistral, Google), so any shared pattern cannot be attributed to the Qwen3 generation; Gemma-3-12B also forms a same-family rung below the Gemma4-26B/31B models in Table 1.

Method: we compare SAER ceilings, not gains. The no-retrieval floors of small models differ by up to 7× (Llama-3.1-8B 2/60 vs. Qwen3-8B 14/60), which inflates the no_ret→saer_cold gain in a

[1]The two models named as future work in an earlier draft (Mistral-7B, Gemma-2-9B) have been retired from our serving platform; we substitute their in-family successors (Ministral-8B, Gemma-3-12B) at the same parameter tier. The OpenRouter client path does not forward the `temperature` parameter to the API (a known harness limitation shared by all OpenRouter profiles); the models therefore run at their provider default temperature.

Table 18: Δ differences the percentages as printed (one decimal for the three added models; Table 1 rounds to integers). Capacity de-aliasing: three cross-family sub-frontier models vs. the Qwen3-8B anchor. SPR is strict majority vote over 3 seeds. We compare the `saer_cold` ceiling (bold), not the gain Δ: no-retrieval floors differ up to 7$\times$ across small models, which inflates Δ and makes it uninformative about capacity. `ps` = parse-success rate (confirms the ceiling is a knowledge signal, not a format artefact; saer_cold ps should be $\approx$100%). McNemar reports b/c (saer_cold-only / no-retrieval-only passes) and the two-tailed p; $^{**}p<0.01$, $^{***}p<0.001$. The three new models' cells are runtime verdicts, not the uniform re-validation of §4.3; see the scoring caveat below.

Model	no_ret (floor)	**saer_cold (ceiling)**	ps no/saer	Δ	McNemar b/c	p
Qwen3-8B (anchor)	14/60 (23%)	**26/60 (43%)**	100/100	+20.0	15/3	0.0075^{**}
Llama-3.1-8B	2/60 (3.3%)	**7/60 (11.7%)**	100/100	+8.4	5/0	0.063
Ministral-8B	13/60 (21.7%)	**23/60 (38.3%)**	100/100	+16.6	11/1	0.0063^{**}
Gemma-3-12B	16/60 (26.7%)	**33/60 (55.0%)**	100/100	+28.3	18/1	$<0.001^{***}$

way uninformative about capacity. We therefore compare the `saer_cold` SPR *ceiling*: if the three new models cluster with Qwen3-8B (26/60) *below* the four frontier primary models (41–52/60), the residual is not specific to the Qwen3 generation. We phrase the criterion this way deliberately. An earlier version read 41/60 as a *capacity* floor; that reading does not survive our own Table 1, where Gemma4-26B-A4B — 4B active parameters, fewer than any model in this panel — reaches a cold-start ceiling of 48/60, and Gemma4-31B reaches 53/60. Within the Gemma family the ordering inverts outright: Gemma-3-12B (12B dense) ceilings at 33/60 against 48/60 for a model with three times fewer active parameters. What this panel can de-alias is family, not capacity. We additionally report `parse_success` (the fraction of tasks whose plan parsed into executable commands) to confirm the ceiling reflects domain knowledge rather than output-format compliance. The five primary-study models are format-compliant under no retrieval (parse-success $\approx$100%, verified on the canonical per-seed files), so for them the no-retrieval baseline is not a format-compliance artefact. Table 18 gives the three models' results.

Verdict. Llama-3.1-8B, Ministral-8B, and Gemma-3-12B reach `saer_cold` SPR of 7/60 (11.7%), 23/60 (38.3%), and 33/60 (55.0%) respectively — all below the four frontier primary models (41–52/60). Against the fifth, Qwen3-8B (26/60), the comparison is not uniform: on the re-scored ceilings below (7, 26 and 34 of 60), Llama-3.1-8B sits below it, Ministral-8B level with it and Gemma-3-12B above it. That is the de-aliasing claim in full — two non-Qwen families land at or under the Qwen3-8B level, so the residual is not an artefact of the Qwen3 generation — and it is all this panel establishes. It does not establish a capacity threshold, and we do not claim one. The 41/60 figure is the minimum over the four frontier models, not over all five and not a floor that parameter count predicts: the Gemma4 replication panel puts a 4B-active model at 48/60 and a 31B model at 53/60 (Table 1), both above it, and inverts the within-family ordering against Gemma-3-12B's 33/60. Within this panel the 12B model does out-rank the two 8B models (33/60 vs. 7–23/60), which is consistent with a capacity gradient but, against the Gemma4 rows, plainly not sufficient to establish one.[2]

Scoring caveat. The three added rows are scored from the verdicts the harness recorded at runtime, not from the uniform re-validation of §4.3 that produces every other table in this paper; the Qwen3-8B anchor row is quoted from the re-scorer. We flag the mismatch rather than leave it to be discovered. Re-scoring the same per-seed files with the current validator (the path taken by `scripts/eval/mcnemar_table.mjs --rescore`) moves the no-retrieval floors to 3/60, 15/60 and 20/60 and the `saer_cold` ceilings to 7/60 (11.7%), 26/60 (43.3%) and 34/60 (56.7%) for Llama-3.1-8B, Ministral-8B and Gemma-3-12B, with McNemar b/c of 5/1, 14/3 and 17/3 (p=0.219, 0.0127, 0.0026). What this section claims is unchanged: all three re-scored ceilings still fall below the four frontier models' 41/60, and Gemma-3-12B's 34/60 still sits above Qwen3-8B's 26/60, so the de-aliasing reading and its one exception both hold under either scoring. What moves is the absolute rates and the nominal significance stars (Ministral drops from $p<0.01$ to $p<0.05$, Gemma-3-12B from $p<0.001$

[2]Llama-3.1-8B's near-zero no-retrieval floor is 2/60 with parse-success at 100% under both conditions, which rules out one competing explanation — that the floor is a formatting failure — and only that one. We did not run the failure-mode breakdown of Appendix T.1 on the three sub-frontier models, so whether the residue is missing knowledge or something else (a plan that parses and still never produces the goal) is untested here; we compare ceilings rather than no-retrieval gains because a floor this low makes the gain uninformative, not because its cause is settled.

to $p<0.01$), which carry no weight here in any case: this panel is excluded from the confirmatory family of Appendix G and its p-values are descriptive.

S Live Execution and Combat: Detailed Results

Table 19 reports the per-level live-execution pass rates discussed in Appendix H. Live completion is governed by an execution-reliability axis (Mineflayer sync/window races; long embodied mine–smelt–craft chains) that floors both conditions equally at the L2 tier, leaving no headroom for the plan-quality difference SAER supplies; the figures are consistent with the bottleneck being severe but cannot measure it, since both arms floor together; as §1, Appendix H and §6.2 also state, this axis corroborates nothing either way. Unlike the static evaluations, this live study runs the runtime `PlanValidator` with one gap-repair regeneration before execution, applied identically to both conditions; since the L2 floor is symmetric (2/21 under both `no_retrieval` and `saer_cold`), we do not read this as directional, though per-trial repair rates were not logged.

Failure-mode audit (Appendix H). Of the 42 `no_retrieval` trial failures, 33 (79%) carry an explicit Mineflayer synchronisation error, 4 time out, 3 abort with no item gained, and 2 are prerequisite-chain gaps. The 46 `saer_cold` failures are distributed differently: 25 are prerequisite-chain gaps, and 21 of those report the same missing `wooden_pickaxe`. We traced that cluster to a runtime defect rather than a planning one. A plan step whose command list bundles several `craftRecipe` calls was judged successful whenever any one of its outputs appeared in the inventory, so a final craft dropped by a window-sync race was recorded as complete and never retried; the missing tool then surfaced at the next mining step. The defect is triggered by the multi-command step format that SAER's demonstrations induce and not by `no_retrieval`'s one-command steps, which give every craft its own verification point. Splitting bundled steps to one craft each removes the symptom in a three-task pilot (`craft/iron_door`, `craft/cauldron`, `craft/iron_leggings` $\times$ 2 conditions $\times$ 3 seeds); the fix is confined to the live runtime and does not touch the static evaluation of Table 1. A full re-run under the fix was not completed, so Table 19 reports the pre-fix results and we do not claim a corrected live ranking — only that a substantial share of the `saer_cold` live deficit is an execution-layer artefact. Plan-length differences are not one-directional: `saer_cold` plans are longer at L1 (5.4 steps against 4.3 for `no_retrieval`) but shorter at L2 and L3 (9.7 and 12.2 steps against 12.6 and 16.5).

Table 20 gives the per-condition combat-domain results (design §4.4; discussion Appendix I). Two independent runs agree: retrieval of any kind lifts majority-vote SPR from 0–1/5 to 3/5, and the three retrieval conditions land on the *identical* three tasks (`kill_creeper`, `kill_enderman`, `kill_slime`); `kill_blaze` and `kill_wither_skeleton` fail under every condition and every seed on the runtime verdicts quoted here; re-scoring moves `kill_blaze` (below). That agreement is largely fixed by construction: with the five near-duplicates excluded the pool holds four demonstrations, so at $k=3$ any two retrieval conditions share at least two of their three. The probe therefore supports the existence effect on the combat subgroup and supplies no evidence on retrieval quality. No paired test is reported because none can be informative here: with 5 tasks, a perfect 5–0 discordant split gives $p=0.0625$ under the two-sided exact binomial used in the main text, above $\alpha=0.05$ before the Bonferroni correction of Appendix G, and the observed splits are 3–0 at best.

Two contamination controls apply. Five of the nine combat demonstrations target the same five mobs as the held-out tasks and differ only by slug prefix (`combat/creeper` vs. `combat/kill_creeper`), so the slug-level non-overlap check of §4.1 does not flag them, and the Jaccard overlap check of Appendix M covers only the 44 CRAFT tasks. Both runs below therefore pass `--exclude-slugs` to remove those five templates from the retrievable pool; with them left in, every retrieval condition reaches 5/5 and the comparison is uninformative. Separately, the combat pool originally assumed a warm inventory without acquisition commands; **R3** uses that original pool and **R4** the corrected cold-start chains. The three retrieval conditions agree across rounds (3/5 each); `no_retrieval` reads 1/5 in R3 and 0/5 in R4, and we quote R4, the corrected pool. The data-quality correction therefore does not change the conclusion.

Run-time versus re-scored verdicts. Like the capacity and thinking-mode panels named in §4.3, the combat probe quotes verdicts recorded at runtime rather than the uniform re-validation, and we flag it here rather than leave it for a reader to discover. Re-scoring the same per-seed files

Table 19: Live execution (Minecraft 1.20.4, DeepSeek-V3, 21 held-out tasks × 2 conditions × 3 seeds = 126 trials, 300 s timeout, self-built tool chain). *Strict* = goal item obtained; *Prereq* = goal's direct ingredients assembled (terminal craft excluded). Trial-level pass rate.

	no_retrieval		saer_cold	
Level	Strict	Prereq	Strict	Prereq
L1	18/21 (86%)	18/21 (86%)	15/21 (71%)	18/21 (86%)
L2	2/21 (10%)	2/21 (10%)	2/21 (10%)	2/21 (10%)
L3	1/21 (5%)	1/21 (5%)	0/21 (0%)	0/21 (0%)
Total	21/63 (33%)	21/63 (33%)	17/63 (27%)	20/63 (32%)

Table 20: Combat domain, four conditions × 5 held-out tasks × 3 seeds, with the five same-mob demonstrations excluded from the retrievable pool. **R3** uses the original combat pool, **R4** the corrected cold-start acquisition chains. "mv" is the per-task majority vote out of 5; "mean" is the pooled per-seed pass rate over 15 trials. Upper block: the verdicts recorded at runtime; the main text quotes the **R4** column of that block, so its "`no_retrieval` 0/5" is the R4 figure and the R3 value is 1/5. Lower block: the same files re-scored with the current validator (`scripts/eval/rescore_combat.mjs`).

	R3 (original pool)		R4 (corrected)	
Condition	mv	mean	mv	mean
Recorded at runtime (as quoted in the main text)				
no_retrieval	1/5	20.0%	0/5	13.3%
random	3/5	53.3%	3/5	46.7%
BM25	3/5	53.3%	3/5	60.0%
saer_cold	3/5	46.7%	3/5	53.3%
Re-scored with the current validator				
no_retrieval	1/5	20.0%	0/5	13.3%
random	3/5	66.7%	3/5	60.0%
BM25	3/5	60.0%	3/5	60.0%
saer_cold	**4/5**	73.3%	**4/5**	66.7%

with the current validator (`scripts/eval/rescore_combat.mjs`, shipped in the code and data release) moves one task: `combat/kill_blaze` (L3) passes under `saer_cold` in both rounds, so `saer_cold` reads 4/5 rather than 3/5, and the pooled means rise correspondingly (Table 20, lower block). `no_retrieval` is unchanged in both rounds (1/5 in R3, 0/5 in R4). The direction of the correction favours the method, so the main text's figures are the conservative ones; and the claim they support — that at n=5 the probe establishes *existence* on the combat subgroup but cannot *rank* retrieval variants — is unaffected, since a margin of one task out of five does not separate three conditions at this sample size. We report both sets rather than silently substitute the more favourable one.

T PlanValidator Pseudocode: Scorer and Runtime Validator

§3.5 of the main text notes that the precondition logic exists twice: once as the runtime component that gates execution, and once reimplemented — single-pass and repair-free — as the scoring criterion for the static experiments. Because the two are not identical, and because only one of them produced the reported numbers, we give both rather than a single idealised sketch.

Algorithm 1 is the one that matters for every table in this paper. It is the `validatePlanCorrectness` function of the static evaluation harness, which the re-scoring script imports directly in order to re-validate every stored seed plan (Appendix N). Algorithm 2 is the deployed `PlanValidator` of the agent itself: it runs in the live agent and is exercised by the live study of Appendix H, never by the static evaluations. Both file paths are given in the code release.

```
Algorithm 1: static scoring criterion  (validatePlanCorrectness)
```

```
Input:  steps[]  -- ordered step array parsed from the model's plan
        inv0     -- starting inventory ([] for the cold-start benchmark)
        goal     -- task goal item and required count
Output: {strict_pass, pvr, issues}

 1. if steps[] is empty: return {strict_pass:false, pvr:false}
 2. A <- multiset(inv0)        // available items; NEVER decremented
 3. issues <- {} ; precsMet <- true ; goalHit <- false
 4. for each step s in steps[]:
 5.   C <- commands(s)          // action_cmd | command | freeformGoal
 6.   (a) declared goal achievement
 7.     if some p in produces(s) has p.item == goal
 8.        and not unrecipedCraftClaim(p, C):  goalHit <- true
 9.   (b) declared preconditions -- items only
10.     for each p in preconditions(s):
11.       if type(p) in {access, station}: continue   // assumed available
12.       if not satisfiedBy(A, p):                   // tool tier inclusive
13.         precsMet <- false ; issues += missing(p)
14.   (c) implicit checks, over a within-step accumulator W <- A
15.     for each cmd in C, in order:
16.       if cmd = collectBlocks(b) and getRequiredTool(b) unsatisfied in W:
17.         issues += tool-gap
18.       if cmd = craftRecipe(x,n) and NO recipe variant of x is
19.          satisfiable from W at ops = ceil(n / outputCount):
20.         issues += ingredient-gap
21.       if cmd = smeltItem(y,n), y correctly named, and W(y) < n:
22.         issues += smelt-input-gap
23.       if cmd = smeltItem(z) with z in {iron,gold,copper}_ore:
24.         issues += naming-gap                      // strict only
25.       W <- W + inferProduces(cmd)
26.   (d) commit step outputs
27.     O <- declared produces(s) UNION inferProduces(cmd) for cmd in C
28.     O <- O minus every unrecipedCraftClaim item
29.     A <- A + O                                    // no consumption
30. if not goalHit and some x in A with x == goal and count >= goalCount:
31.   goalHit <- true
32. if not goalHit: issues += goal-not-produced
33. return {strict_pass: (issues empty) and goalHit,
34.         pvr:         precsMet and goalHit}
```

Four properties of Algorithm 1 are worth reading off it directly, because each shapes a reported number.

The scorer does not model consumption, by design. A is only ever extended (line 29); nothing is subtracted when a step uses a material. The metric asks whether the plan gets the prerequisite *structure* right — does it acquire each input before it is needed, name it correctly, and reach the goal — and deliberately leaves the material budget to the runtime, where it belongs. That division of labour is implemented, not aspirational: when a craft fails on the bot, `skills.js` emits `Insufficient resources`, `_classifyError` maps it to `MISSING_ITEM`, and `_routeRecovery` resolves the shortfall through `RecipeResolver.resolveDependencyChain`, pushing an acquisition subtask that `TaskManager` executes before retrying the step. Quantity is a closed loop at execution time, so a static scorer that also charged material budget would be measuring the planner for something the architecture assigns elsewhere. Local quantity errors are still caught where they bite — line 19 requires each craft's ingredients to be present at the point of use, scaled by $\lceil n/\text{outputCount} \rceil$. What is not enforced is the aggregate budget across a plan; that is the hand-off to the runtime replenisher described above, and it is the sense in which a demonstration or a plan can be "materially short" (Appendix U).

Stations and access preconditions are assumed satisfied (line 11). A plan is never failed for lacking a crafting table or furnace. This is deliberate — the static setting cannot verify world state — and it is the single largest respect in which the scorer is more permissive than the runtime validator, which does check stations by name.

Strict and PVR diverge by construction. `precsMet` is set false only in block (b), by *declared* preconditions; `issues` additionally collects the implicit gaps of block (c) and the naming gap of line 24. PVR therefore measures exactly what §4.3 says it measures, and the SPR–PVR gap reported

throughout the paper is mechanically the count of tasks whose plans declare a coherent precondition chain but fail an implicit tool, ingredient, quantity or naming check.

The infeasible-task rule is explicit. `unrecipedCraftClaim` (lines 8 and 28) refuses to credit a declared `produces` whose only supporting command is a `craftRecipe` call for an item with no recipe in `minecraft-data` 1.20.4. This is what makes `craft/iron_horse_armor` fail in all 102 occurrences (Appendix F) rather than passing on a hand-declared output. Apart from that rule, declared `produces` are trusted (lines 7 and 27). In the 16 primary-study runs they appear in no plan of a no-retrieval arm and in at most 12 of 180 plans of a retrieval arm, where the model copies a demonstration's `Produces:` line, so we re-scored those runs with every declared field removed: one majority-vote outcome changes in each GPT-4o retrieval arm (BM25 42→41, `saer_cold` 41→40), none in any other primary-study arm, and no verdict of the pre-specified family moves.

```
Algorithm 2: runtime PlanValidator  (deployed agent; live study only)
Input:  steps[], inv0
Output: {valid, gaps}

1. V <- multiset(inv0)                      // virtual inventory
2. for each step s in steps[]:
3.   P <- normalise(preconditions(s))
4.        + inferImplicitPreconditions(cmd) for each cmd in commands(s)
5.   if s contains a smeltItem command:
6.     drop fuel terms from P               // smeltItem selects fuel itself
7.   for each p in P:
8.     if not satisfied(p, V): gaps += p    // access: always satisfied
9.                                          // station: by name
10.                                         // item: name + block-drop alias
11.                                         //       + inclusive tool tier
12.  V <- V - {p in P : p is an item, not a tool, not fuel}   // consumption
13.  D <- declared produces(s) ; I <- inferProduces(cmd) for cmd in commands(s)
14.  if I is non-empty:                     // executability constraint
15.    for each p in D that is an item with p.name not among names(I):
16.      gaps += unsupported-produce(p) ; drop p from D
17.  V <- V + dedup(D + I)
18. return {valid: gaps is empty, gaps}
```

Algorithm 2 differs from Algorithm 1 in four ways, all of them consequences of having a real world to act in: it *does* track consumption (line 12, tools and fuel exempt), it checks station preconditions rather than assuming them, it exempts fuel on smelting steps because `smeltItem` sources its own fuel, and it enforces an executability constraint (lines 14–16) that rejects a declared `produces` no command in the step can generate. On failure the enhancer issues one targeted gap-repair regeneration before execution; that repair path exists only here, and §3.5 and Figure 1 state where it applies. Tool-tier compatibility is inclusive in both algorithms: a higher tier satisfies any lower requirement. In both, implicit requirements resolve against the `minecraft-data` 1.20.4 recipe database and the tool-tier lookup `RecipeResolver.getRequiredTool`.

We report the divergence rather than present one algorithm as both. No reported number depends on Algorithm 2: the static tables are produced by Algorithm 1 alone, and the live study of Appendix H — the only place Algorithm 2 runs — is reported as a framework-bound limitation rather than as evidence for the method.

T.1 Failure-Mode Breakdown

§6.1 characterises what the failures are; Table 21 counts them. The scorer listed above emits six typed issues, so the breakdown is mechanical rather than hand-annotated: we re-score every stored seed plan with the same fixed validator that produces Table 1, take the majority vote over three seeds, and attribute a failing task to every issue class present in at least two of its failing seeds. The script asserts that each re-scored SPR equals its published Table 1 value before reporting, so a drifted run identifier fails loudly rather than silently scoring the wrong file.

What is shared. Each model's failing tasks partition into three groups: those carrying at least one unacquired-prerequisite issue (precondition, ingredient, tool or smelt-input), those failing only because the goal item is never produced, and those the majority rule cannot attribute at all. Without retrieval the first group holds 41/46 (Qwen3-8B), 31/37 (GPT-4o), 31/35 (Claude Sonnet 4.5), 39/41

Table 21: Failure modes of the five primary-study models, by issue class. *Fail* is the number of tasks failing the majority vote (60−SPR). Classes, in column order: unsatisfied declared *precond*ition, missing craft *ingred*ients, required *tool* unavailable, *smelt* input short, version-specific *naming*, goal item *no*t produced, and *unattrib*uted — the task failed in at least two seeds but no single class was present in two of them, so the majority rule assigns it none. A task may exhibit several classes, so the six class columns over-count; the partition quoted in the text does not. Reproduce with `node scripts/eval/failure_mode_counts.mjs`.

Model	Fail	precond.	ingred.	tool	smelt	naming	no goal	unattrib.
no_retrieval								
Qwen3-8B	46	8	39	24	0	0	6	0
GPT-4o	37	26	3	29	0	0	11	1
Claude S. 4.5	35	27	30	2	0	0	7	0
Qwen2.5-Max	41	30	14	33	1	0	6	0
DeepSeek-V3	27	14	10	3	0	0	6	2
saer_cold								
Qwen3-8B	34	9	26	11	0	0	3	1
GPT-4o	19	11	1	12	0	0	2	1
Claude S. 4.5	9	2	7	1	0	0	2	0
Qwen2.5-Max	11	2	7	3	0	0	2	1
DeepSeek-V3	8	3	2	3	1	0	2	1

(Qwen2.5-Max) and 20/27 (DeepSeek-V3) — 74–95% of failures. The shared signature is therefore an *unacquired prerequisite*, not a malformed or off-goal plan. Retrieval shrinks both groups by similar proportions: the prerequisite group falls from 162 to 67 tasks in total (to 30, 16, 8, 8 and 5), and the goal-only residue from 21 to 10 (5/5/4/2/5 → 3/2/1/2/2).

The third group is small but real, and we name it rather than fold it into the second: 0/1/0/0/2 tasks without retrieval and 1/1/0/1/1 with it fail in a majority of seeds while no *single* class reaches that majority, because the three seeds fail for different reasons. A two-way split would absorb them into "goal item never produced", which they need not exhibit at all; the script asserts the three-way partition sums to the failing-task count and that the goal-only group never exceeds the goal-not-produced column, so this cannot silently reappear.

What is not shared. *Which* prerequisite is missing does not generalise. Ingredient gaps run 39 tasks on Qwen3-8B against 3 on GPT-4o; tool gaps run 33 on Qwen2.5-Max against 2 on Claude Sonnet 4.5. An earlier draft of this paper asserted the same three failure modes at every scale; the counts do not support that, and §6.1 now states only the shared prerequisite signature.

A withdrawn mode. The naming-gap column is zero in every model and both conditions. The scorer's naming check fires only on `smeltItem` applied to an ore name that Minecraft 1.17+ replaced with a raw-material drop (Appendix T), and no majority-vote failure in the primary study trips it. Version-specific naming was listed as a failure mode in an earlier draft on the strength of development-time observations; it is not observable in the reported runs and we no longer claim it. Whether it would reappear under a broader naming check is untested.

U Two Analyses of Demonstration Quality

The demonstration bank is the artefact the retrieval conditions differ over, so its own quality is worth measuring rather than assuming. Two analyses do so. Both are offline, require no model calls, and ship as runnable scripts; both are post-hoc and sit outside the confirmatory family of Appendix G, with nominal p throughout. The first asks whether demonstration quality could explain SAER's edge over BM25 — it cannot, because SAER is handed the worse demonstrations and scores higher anyway. The second isolates a transmission mechanism for the correctness axis of §6.1, on a baseline of zero in four of the five models: planners copy numeric quantities out of the demonstrations they are given.

Does Retriever Choice Change Demonstration Quality?

If SAER and BM25 differed in how often they retrieve a materially broken demonstration, that difference would confound §5.2's reading of SAER's edge as a *relevance* effect. Retrieval is deterministic given the task and the bank, so the question is decidable exactly rather than by argument.

What counts as materially broken. Re-walk a demonstration from an empty inventory over an inventory that is *debited* as well as credited — each produced unit spendable only once, and a `craftRecipe(x,n)` call charged $\lceil n/\text{outputCount} \rceil$ recipe applications, which is the conversion the deployed `skills.js` performs — and some entries reach a craft they cannot afford. On the evaluation-time bank this holds for 11 of the 25 crafting, 10 of the 21 collecting and 3 of the 9 combat demonstrations. The static scorer does not charge for the aggregate material budget across a plan (Appendix T), so these defects move no credit-only number reported in this paper; they matter here only as a property of the pool both retrievers draw from.

The comparison. We recomputed both selections exactly, with no model calls (`scripts/eval/retriever_defect_exposure.mjs`, shipped; it aborts if the `setRecipeResolver` defect has regressed, since an inert resolver would collapse SAER to keyword matching and make the comparison vacuous). The two retrievers share only 1.00 of their 3 selections per task on average, so there is ample room for them to differ.

They do differ, **and the difference runs against SAER** (Table 22). SAER draws 73 of 180 defective demonstrations against BM25's 54, and the gap is concentrated at the top-ranked position (§3.4): SAER's top-ranked demonstration is materially broken on 22 of 60 tasks against BM25's 8, and the paired split runs entirely one way — SAER-only on 14 tasks, BM25-only on *zero*, two-sided p=0.00012. The per-task totals favour BM25 in the same direction but do not reach significance (26 versus 15 tasks, sign p=0.117).

Table 22: Materially broken demonstrations retrieved over the 60 frozen tasks at k=3, on the evaluation-time demonstration bank. Post-hoc, outside the confirmatory family of Appendix G; nominal p. ‡ = the one category where the direction reverses, on n=5 tasks drawing from a nine-example pool in which five are near-duplicates of those tasks (§4.4), retrieved over in full here as in Table 1; we do not read a direction off it.

	SAER	BM25
Defective demonstrations retrieved	73/180	54/180
Mean per task (of 3)	1.22	0.90
CRAFT (n=44)	1.05	0.89
COLLECT (n=11)	1.73	0.55
COMBAT (n=5)‡	1.60	1.80
Top-1 defective	22/60	8/60
Top-1 paired: SAER-only 14, BM25-only 0, p=0.00012		

Why, and what it implies. The mechanism is that SAER is working as designed. Its dominant dimension is ingredient overlap at α=0.50, which matches material tier by construction, and the benchmark's L2/L3 tasks are iron- and diamond-tier tools. In this bank, tier and brittleness are correlated: 8 of the 10 iron/diamond/gold-tier crafting demonstrations are materially short, against 3 of the other 15. SAER therefore retrieves the same-tier demonstration — the relevant one — and in this pool the relevant one is usually the broken one. The natural completion is that BM25, matching lexically, is less tightly locked to tier, but we did not measure its tier-match rate, so that half is a conjecture: the counts above are equally consistent with BM25 simply selecting more diffusely. The claim we do make is narrower and the data carries it: the defects are concentrated in one tier, and all eight of them are the same hardcoded stick constant, diagnosed below, which the later corrected bank revision repairs. Separating "property of the bank" from "property of the scorer" would need this exposure count recomputed on the corrected revision, which the shipped scripts support through the `CRAFTING_BANK` switch (Appendix M) and we have not run.

The consequence for §5.2 is the opposite of a confound. SAER reaches 49/60 against BM25's 42/60 *while* being fed the materially worse demonstrations, so demonstration quality cannot be what generates SAER's advantage, and the relevance reading of §5.2 survives this particular check. Under the credit-only scorer these shortfalls cost no pass, so they can work against SAER only under the debited scorer that follows. The relevance reading does not survive every scoring choice: the static scorer credits produced items without debiting them (Appendix T), and if material-balance violations are instead scored as failures, `saer_cold` reads 33/60 against BM25's 28/60 and the contrast loses its nominal star (p=0.0156 → 0.125), while BM25 (28) falls below random (31), inverting the ordering of §5.2 (p=0.375). Pooled over the five primary models' arms and the ablation — the Gemma4 rows of Table 1 are not included — 84 of 413 majority-vote strict passes (20.3%) carry the violation in a majority of their passing seeds, and 94 (22.8%) lose the majority vote once the budget is charged. The second set contains the first: the extra ten pass twice on credit, once cleanly, and fall below 2 of 3 when the violating seed is withdrawn. The 94 are what Table 23 and the debited columns of Table 2 and of the primary rows of Table 1 charge; the 20.3% is the narrower, more conservative count (`scripts/eval/measure_consumption_leniency.mjs`, shipped). The leniency is not symmetric across conditions, which is why it matters here rather than as generic evaluator noise: charging the material budget removes 0 of `no_retrieval`'s 19 passes, 7 of `random`'s 38, 14 of BM25's 42 and 16 of `saer_cold`'s 49. A scorer that never debits can only credit plans that produce things, and the retrieval arms produce more.

What survives the stricter rule and what does not is set out in Table 23. *Existence* stays family-wise significant (random vs. no retrieval, b=13, c=1, p=0.0018). All five per-model SAER effects stay positive, but three rather than four clear α'=0.0038: DeepSeek-V3 (p=0.0309) becomes nominal and Qwen3-8B stays nominal (p=0.0127), and BM25 vs. no retrieval drops from $<$0.001 to 0.0352. The relevance increment does not survive at all, which is why §5.2 reports it as directional rather than established. An earlier version of this appendix asserted that every per-model effect stayed *significant*; the per-model recomputation below does not support that, and the claim is withdrawn. Tables 1 and 2 state both scorings side by side; the ablation's three effects over `no_retrieval` shrink by 37–61% in tasks under the stricter rule (19→12, 23→9, 30→14). The replication panel, computed separately so that the pooled count above is unchanged, loses its credit-only gains under debiting (bottom rows).

Table 23: Per-model paired tests with material-balance violations scored as failures. Same runs, same majority vote, same three-seed rule; only the pass predicate changes. Reproduce with `node scripts/eval/measure_consumption_leniency.mjs --mcnemar`, and the replication rows by adding `--replication`; those rows lie outside the pre-specified family, so their verdicts are descriptive.

Comparison	SPR (/60)	b/c	p	Verdict
existence: `saer_cold` vs. `no_retrieval`				
Qwen3-8B	13→24	14/3	0.0127	nominal
GPT-4o	20→34	15/1	0.00052	family-wise
Claude S. 4.5	22→35	14/1	0.00098	family-wise
Qwen2.5-Max	19→33	15/1	0.00052	family-wise
DeepSeek-V3	25→35	14/4	0.0309	nominal
ablation on Qwen2.5-Max				
`random` vs. `no_ret`	19→31	13/1	0.0018	family-wise
`bm25` vs. `no_ret`	19→28	12/3	0.0352	nominal
`bm25` vs. `random`	31→28	1/4	0.375	n.s.
`saer_cold` vs. `bm25`	28→33	6/1	0.125	n.s.
replication panel: `saer_cold` vs. `no_retrieval` (outside the family)				
Gemma4-26B	35→28	4/11	0.118	n.s.
Gemma4-31B	37→37	6/6	1.000	n.s.

It also sharpens the closing claim that demonstration quality is the next lever — under the debited scorer, repairing the bank should help the relevance-ranked retriever more than the lexical one, because the relevance-ranked one is the one currently being handed the broken examples.

Status. The detector resolves every ambiguity toward "no defect", so both columns are lower bounds; the comparison is between two retrievers scored by the same detector over the same bank, which is what the confound question asks.

Quantity Inheritance from the Demonstration Bank

Applied to the bank rather than to the retrievers, the same detector asks whether the demonstrations are materially executable at all. They are not, uniformly, and the shortfalls turn out to be traceable in the models' own output.

The bank. Every failure counted above is the same kind: a hardcoded quantity in the shared opening — collect logs, craft planks, craft sticks, craft a wooden pickaxe — that does not scale with the chain the demonstration then walks. `skills.js` reads these arguments as output-item counts and converts them to $\lceil n/\text{yield} \rceil$ recipe applications, so the constant fixes the budget exactly. The two banks fail on different constants. In the crafting bank, 9 of the 11 failures are *sticks*: the `stick` argument takes only the values 2 and 4 in this revision, and `craftRecipe('stick', 4)` is a single application yielding four, whereas a demonstration that crafts a wooden, then a stone, then an iron tool needs six. `crafting/iron_axe` is representative: it reaches its final craft with zero sticks against the two an iron axe requires. In the collecting bank 8 of the 10 failures are *planks*: 9 of its 17 `oak_planks` calls ask for 4, against 3 of 25 in the crafting bank, and four planks cannot cover a crafting table (4), sticks (2) and a pickaxe (3). The later corrected revision, supplied alongside the evaluated one, raises the stick constant to 6 and 8 — among other edits, which Appendix M describes from the ingredient-set side — and that is what takes its material-shortfall count to 1 of 25 rather than 11. The recipe graph is available to neither the demonstration author nor, as the next paragraph shows, the model that reads it.

The transfer. Because the two banks carry different constants, the constant a model emits is a fingerprint of what it read. Table 24 counts the `oak_planks` argument in every generated plan for the 11 COLLECT tasks, which retrieve from the collecting bank. Under `no_retrieval` four of the five primary models emit the sufficient 12 in *every* trial — 0 of 17, 0 of 18, 0 of 8 and 0 of 15 uses of the insufficient 4. The fifth is the exception that bounds the argument: Qwen3-8B parsed only 2 COLLECT plans without retrieval and both used 4, so its baseline is 2/2 rather than zero, and the contrast below is a four-model one. Under retrieval the insufficient value appears: 10 of 12 trials on Qwen2.5-Max, 13 of 13 on Qwen3-8B, 4 of 15 on DeepSeek-V3 and 2 of 17 on GPT-4o. The same models keep emitting 12 on CRAFT tasks. The crafting bank is not uniform either — its 25 planks calls read 12 eleven times, 8 ten times, 4 three times and 20 once — so what separates the two categories is how often the *insufficient* 4 appears: 3 of 25 calls against the collecting bank's 9 of 17. On CRAFT the models therefore emit a sufficient value that a majority of their retrieved demonstrations also carry; on COLLECT they emit the short one that a majority carries. Random retrieval transmits it too — 6 of its 12 COLLECT trials on Qwen2.5-Max, a condition Table 24 does not tabulate — as it must, since it draws from the same bank — so this is a property of the demonstration pool, not of SAER's ranking.

Table 24: `oak_planks` count emitted on COLLECT tasks (all seeds, per trial). The retrieved collecting bank says 4, which is materially short; 12 suffices. Claude Sonnet 4.5 overrides the demonstration in every trial.

	`no_retrieval`		`saer_cold`	
Model	n	$\% = 4$	n	$\% = 4$
Qwen3-8B	2	100	13	100
GPT-4o	17	0	17	12
Claude S. 4.5	18	0	16	0
Qwen2.5-Max	8	0	12	83
DeepSeek-V3	15	0	15	27

What this establishes. It supplies the transmission mechanism behind the correctness axis of §6.1, which the 35-task contrast of §6.1 shows only through outcomes — SAER trails random until broken

L2 demonstrations are repaired. Here it is operating, on a baseline of zero in the four models that have one: the planner reproduces a numeric argument it read, and the argument is wrong. It also explains why the architecture assigns quantity to the runtime rather than to the planner — these models demonstrably do not check a copied constant against the recipe graph, so something downstream has to. The models differ: Claude Sonnet 4.5 overrides the demonstration in all 16 trials, Qwen3-8B in none of 13. None of the copied quantities changes a credit-only number, since that scorer does not charge for them (Appendix T), which is why they survived to be found.

We claim no dose–response from this: it does not follow that a bank's defect rate predicts how often plans in the corresponding category are themselves short, and plan length is an uncontrolled confound throughout, since longer chains offer more chances to under-budget regardless of what was retrieved. What the fingerprint does establish, on a baseline of zero in four of the five models, is that these planners copy numeric arguments out of retrieved demonstrations without checking them against the recipe graph, and that a single wrong constant in one demonstration file propagates to most trials of every task that file serves. It also bounds the bank audit of Appendix E, which verified that each declared precondition is produced by an earlier step: that is a statement about the *presence* of an item, not about the *amount*. The bank is short on amounts, and the audit was not designed to see it.

V Oracle Categories and the Deployment Routing Path

The deployed pipeline classifies a request with an LLM call (`getIntent()`, §3.2) and admits only demonstrations from the matching file. §3.2 enumerates the labels the classifier in the code release emits. The static harness does *not* run that classifier. Instead it reads the ground-truth `category` field of the frozen task list, at two places in `scripts/eval/static_plan_eval.js`: the per-task loop calls `loadTrainingData(task.category)` to pick the demonstration file, and `generatePlan` builds the request object with fields `input: task.prompt` and `intent: task.category`, whose `intent` field is emitted verbatim into the prompt by `FewShotLearning.buildRequestAnalysisSection` as an `Intent Type:` line. All reported numbers are therefore an oracle-category upper bound, and we state this rather than leave it to be inferred from the release.

What it does not bias. Both injection points sit *above* the `switch` over conditions. The request object is constructed once and passed unchanged to every branch, and the demonstration pool is loaded once per task before the branch is selected. Consequently `no_retrieval` carries the same `Intent Type:` line as `saer_cold` — it simply carries no examples — and `random_retrieval`, `bm25_retrieval` and `saer_cold` all sample from the identical, oracle-selected candidate pool. No condition receives a category signal the others lack. The oracle can therefore raise or lower all conditions together, but it cannot move the paired differences between them, and every claim in this paper is stated as such a difference (McNemar on paired per-task outcomes, never on a single condition's absolute rate). The retriever itself never reads the label: `SAERScorer` contains no reference to `intent`, and its four component scores are computed from ingredients, tokens, tool tiers and difficulty alone.

A second candidate, checked and ruled out. The category is one place the frozen task list could leak into retrieval; the difficulty tier is the other, because $\delta \cdot L$ of §3.3 needs a level for the *query* and the task list carries one. It does not come from there. `SAERScorer.estimateLevel` takes only the request string and matches it against four ordered keyword lists, defaulting to 2, and the harness passes it nothing else (`input: task.prompt`). Evaluated against the frozen list's own tier labels it agrees on **20 of 60** tasks — the same score a constant "L2" answer gets, since 20 of the 60 are L2 — with 41 of the 60 estimated as L2 and the confusions running both ways (12 L3 tasks read as L2, 6 L2 tasks as L1). So the fourth dimension is a weak surface heuristic, not a second oracle, and the tier it scores against is not the tier the benchmark stratifies on.

What it does bias. Deployment fidelity. If `getIntent()` misclassifies a request, the wrong demonstration file is loaded and *all* retrieval conditions degrade towards the no-retrieval floor, because none of them can rank an example that was never a candidate. The size of that degradation is a product of two quantities: the classifier's error rate over the 60 tasks, and the conditional loss given an error. We measured the first, which makes the second moot here. Running the deployed classifier over the frozen task list — `determineIntentType` invoked on its shipped prompt and parser, DeepSeek-V3, three seeds, 180 calls, zero API failures — gives **100% routing accuracy**

at the task level (60/60 under the same majority vote used throughout), with no task misrouted; trial-level label accuracy in that run was 99.4%, and every label-discordant trial still routed to the same demonstration file as its ground truth, so its conditional loss was zero. These are figures from one dated run against a live commercial endpoint rather than a deterministic quantity: trial-level counts, and which particular tasks come out seed-unstable, move between runs. We therefore rest the argument on the majority-vote routing result, which is the quantity the oracle bound actually concerns, and on that quantity the bound is tight on this benchmark: no reported number would change under the deployed router.

We claim this for the 60 held-out prompts only. They are short and templated ("craft a wooden sword", "kill a creeper"), so the result bounds how optimistic the oracle is *here*; it is not evidence that the classifier is robust on the free-form chat the deployed agent also accepts, which remains unmeasured. Two further limits are worth stating. The model was served from the first-party DeepSeek endpoint rather than the third-party path of the deployed profile, which serves the same `deepseek-v3` model. And routing accuracy is measured against the oracle `category` field, so it inherits whatever labelling convention that field encodes. The measurement script is `scripts/eval/intent_accuracy.mjs`; it reaches the deployed prompt and parser through the class prototype rather than restating them, so it cannot drift from the shipped classifier.

A structural routing defect, found and fixed. Auditing the classifier against the oracle exposed a defect that the oracle had been masking, and we report it rather than leave it in the release. The classifier's category list folded combat into the movement category (*"EXPLORE — combat, movement, or exploration"*), and a separate hardcoded whitelist inside the response parser enumerated labels that likewise excluded `combat`, coercing any `combat` verdict to the `general` fallback. The intent-to-file map, meanwhile, already contained a `combat` → `combat_examples.json` entry. The two ends therefore disagreed, and `combat_examples.json` was unreachable from the deployed path: a request such as "attack a zombie" was routed either to the collecting demonstrations, through the movement label, or to the crafting demonstrations, through the `general` default. The frozen benchmark contains 5 combat tasks among 44 `craft` and 11 `collect`, so under deployment those five could not have been routed correctly by construction — had the deployed router been in the loop at all, which for the static results it was not. The node-level pipeline figure in the code release records this defect: the label set drawn there is the pre-fix deployment one, and it carries no combat entry.

The code release fixes this: `combat` is now a first-class category with its own description and keywords, movement and exploration are described separately, and the parser's whitelist is derived from the intent-to-file map rather than restated, so the two can no longer drift apart. A regression test (`tests/intent_routing.test.js`, 34 assertions) asserts the invariant directly: every label the map can route is enumerated in the classification prompt, survives the parser without coercion, and does not resolve to the crafting default. §3.2 describes the fixed classifier.

No reported number changes. The fix cannot move any result in this paper, because the static harness never calls `getIntent()` — it reads the `category` field of the frozen task list, and that field was already `combat` for the five combat tasks, which already loaded `combat_examples.json`. The defect lived exclusively on the deployment path. What it does change is the interpretation: before the fix, the combat results were an existence probe under a router that could not have reproduced them in deployment; after it, the deployed router can reach the same demonstration file the evaluation used. The routing measurement above closes the half of this gap that concerns label accuracy: the deployed router reaches the correct demonstration file for all five combat tasks, as it does for all 60. What remains unmeasured is the other half — an end-to-end re-run of the main table under predicted rather than oracle categories, which would also carry the classifier's own latency and failure modes. With routing accuracy at 60/60 that re-run is expected to reproduce the reported numbers, but we report the expectation as such rather than as a result. Neither half bears on the knowledge-bottleneck claims, which are contrastive.

Three branches are nominal rather than studied. The frozen benchmark is 44 `craft`, 11 `collect` and 5 `combat` tasks; it contains no `cook`, `build` or `explore` task, and no result anywhere in this paper exercises those branches. They exist in the router because the agent accepts free-form chat, not because they are part of the evaluated system, and the defect described above is correspondingly a deployment-path defect only. We report it because the code release is an artefact readers may run, not because it bears on a measurement.

Demonstration command validity, disclosed but not repaired. The command validator shipped with the release checked only the crafting and collecting files. Extending it to every routed file — combat and build had never been checked, combat because it was unreachable — reports 25 invalid commands: 13 in the combat file, 9 in the build file and 3 in the crafting file. One defect dominates: 13 uses of `!equipItem`, which is not in the bot command registry (the registered name is `!equip`), spread across the combat and build files. The remainder are 4 uses of `planks` where a specific wood type is required, 2 `!startBuild` calls, 3 `!placeHere` calls naming an item rather than a placeable block (`flint_and_steel`, `water_bucket`), and 3 counts of a single `!attack` call missing its second argument. We report these rather than repair them. The demonstration bank is the frozen artefact behind every reported number, and editing it after the fact would fork the artefact from the results; the validator extension is a script change and touches no data.

Reachability is not uniform, so we measured it. The build file is never retrieved, since no benchmark task carries that category. The three crafting errors are confined to a single demonstration, `crafting/bow`, which the retriever selects for none of the 44 craft tasks. The combat file is the exception: all 9 of its demonstrations contain `!equipItem`, and all 5 combat tasks retrieve at least one of them, so every combat prompt under every retrieval condition carries the invalid command.

Measuring demonstration-defect inheritance. That contamination propagates, and the propagation is measurable — which makes this the one part of this appendix that is a finding rather than a disclosure. Because the defect sits in *every* retrieval condition's prompt and in *no* no-retrieval prompt, the gap between the two arms isolates how often a planner copies a broken command straight out of its demonstrations. Scanning every stored per-seed result file under `scripts/eval/results` (308 files, superseded runs included) for the string `equipItem` in the generated plan of a `combat/` task: it appears in *0* of 505 no-retrieval trials, and in 726 of 1,016 retrieval trials (71.5%) — 72.9% under `saer_cold`, 74.2% under BM25, 61.2% under random. A model given no demonstrations never invents this command; a model given these demonstrations copies it verbatim about three times in four. The rate is the stable quantity here: an earlier, narrower sweep, whose file set is not recoverable, also gave 71%, and the per-condition rates move by at most two points across sweep widths while the no-retrieval count is exactly zero at every width. We therefore ship the scan rather than the counts: `scripts/eval/scan_demo_defect_inheritance.mjs` states its file set explicitly and asserts the zero.

The inheritance is invisible to the metric because the static scorer infers production only from `craftRecipe`, `smeltItem`, collection and `attack` commands (an attack credits five units of the mob's drop, which is how a combat task reaches its goal item, e.g. `gunpowder` for `combat/kill_creeper`); an equip step produces nothing under either spelling, so a wrong one is neither penalised nor rewarded. Consistently, no validation issue in any scanned combat trial mentions `equip`. The reported combat numbers are therefore unaffected, and the probe of Appendix I remains a statement about plan structure. But the plans it scores as correct contain a command that would fail on contact with the bot, which is a concrete instance of the gap between this static metric and execution that Appendix H discusses in the aggregate.

Scope of these fixes. None of them can move a reported number, and the reason is structural rather than argued: the static harness `scripts/eval/static_plan_eval.js` does not import the enhancer module at all. It constructs `FewShotLearning`, `BM25Retriever` and `RecipeResolver` directly, scores plans with its own validator, and carries its own intent-to-file table. The classifier and the routing table described above are reachable only from the deployed agent path.

W A Separate Live Probe: Structured Fact Sharing Between Two Agents

No claim of this paper rests on this appendix. It reports a two-bot live study of the *second* retrieval stage of the same enhancer, kept so that the released system can be read end to end and because the numbers were collected under this project. It exercises none of the components varied in the main study: no SAER retrieval, no plan validator, no executor from Figure 1. The node-level pipeline figure, the per-trial rows, the contact sheets of the 18 screen recordings and the scripts that re-derive every number below ship with the code release (under `evidence/knowledge_reuse/`; the figure under `docs/paper/neurips_ws/figures/`).

The second stage. The procedural stage (SAER) filters demonstrations by intent category, ranks them by ingredient, keyword and level overlap, and places the top three before the planning call. The team stage has the same shape over a different record: when more than one bot is active, a shared store (`SharedMemory`) holds *world facts* (source agent, resource, position, confidence, timestamps, optional expiry); a query filters them by namespace, record type, resource, expiry, age and radius, ranks them by source confidence discounted by distance, and places the result in the consumer's prompt immediately before its single selection call. Neither stage updates any weights. The correspondence is one of shape, not shared code. Only the world-fact write/read path (`recordWorldFact()` / `queryWorldFacts()`) is on the measured route; the store's reservations, supply requests and work ledger are implemented but not measured here.

Design. Three model endpoints (Doubao-Seed-2.0-Mini `doubao-seed-2-0-mini-260428`, a hosted `qwen-max` endpoint, and Qwen3-8B served locally through Ollama as `qwen3:8b`) $\times$ three sharing conditions — none (*normal*), free-form chat (*explicit*), structured fact sharing (*implicit*) — $\times$ two tasks (*geothermal*: collect soul sand from the one valid site; *workstation*: reach a discovered crafting table and craft a wooden pickaxe) $\times$ five trials per cell, 90 trials in all. A scout follows a fixed route and finds the target at its second inspection, so the discovery moment is constant; target identity rotates across attempts and is identical within each normal/explicit/implicit triplet. Under *normal* the consumer has no information from the scout and searches three candidate sites; under *explicit* the scout reports over chat, which the consumer must parse; under *implicit* the consumer reads the structured fact directly. The store is rebuilt from empty for every attempt, and *normal* and *explicit* never touch it. Each trial issues one scripted model call (two under *explicit*: report and interpretation). Before every attempt both agents' inventories and memories are cleared and the terrain reset; teleportation is allowed only before the trial announcement.

Table 25: Cross-agent context reuse (30 trials per condition, pooled across models). Post: mean post-discovery duration (s); Visits: unnecessary consumer-side site inspections; Calls: model calls; Chat: agent-to-agent messages; Cov.: physical objective verified.

Condition	Succ.	Post	Visits	Calls	Chat	Cov.
Normal	30/30	21.58	22	30	0	1.00
Explicit	28/30	18.07	0	60	60	0.93
Implicit	30/30	16.73	0	30	0	1.00

Results. Success is saturated for *normal* and *implicit* (30/30; Table 25), so the signal is overhead. *Implicit* removes all 22 redundant consumer inspections and uses 30 model calls against *explicit*'s 60; both are exact counts over the recorded trials, and the call reduction follows from the protocol rather than from model behaviour, so neither carries a test. The tests belong to post-discovery duration: paired *implicit*$-$*normal* is -4.85 s (bootstrap 95% CI $[-6.68, -3.05]$; 26/30 faster; two-sided sign test $p{=}5.95{\times}10^{-5}$), and *implicit*$-$*explicit* is -1.33 s ($[-2.16, -0.52]$; 24/30 faster; $p{=}0.0014$). *Explicit*'s two failures both occur when the consumer selects a different site from the one the report names. The pooled speed-up is not uniform across endpoints (Table 26): against *normal* it is significant on both hosted APIs (10/10 faster each, $p{=}0.0020$) but not on the locally served 8B model (6/10, $p{=}0.754$), and against *explicit* its interval excludes zero only for Doubao. Intervals are percentile intervals from 20,000 paired bootstrap resamples; because the five attempts per cell reuse two scenario templates, the inference describes the recorded protocol rather than a task population. All contrasts lie outside the confirmatory family of Appendix G.

Integrity. An automated audit over the retained records passes all 18 programmed checks: exactly 90 trials, 5 per cell; all 30 triplets target-matched; 90 valid reset audits and 90 unique per-trial state paths; 60 of 60 *explicit* messages observed and model-generated, against none under *implicit*; a structured-fact create-and-consume trace in all 30 *implicit* trials; and no work-progress teleportation. No prompt in any condition contains the words *duplicate*, *overlap*, *division of labor*, *role assignment* or *assigned role*, so the coordination observed is not instructed. The shipped logs have absolute paths replaced, so integrity checks should use `evidence/CHECKSUMS_AS_SHIPPED.txt`, not the runtime manifest's hashes.

Table 26: Per-model outcomes. I/N/E = *implicit*, *normal*, *explicit*. "Avoided" = excess consumer site visits eliminated relative to *normal*. Differences are implicit minus baseline post-discovery seconds with bootstrap 95% CIs; intervals containing zero are inconclusive.

Model	Success I/N/E	Avoided	Δ vs. N [CI]	Δ vs. E [CI]	Calls I/E
Qwen3-8B (Ollama)	10/10; 10/10; 9/10	8	-2.90 $[-6.01, 0.17]$	-0.05 $[-1.35, 1.28]$	10/20
Doubao-Seed-2.0-Mini	10/10; 10/10; 10/10	6	-5.95 $[-8.74, -3.27]$	-3.48 $[-4.43, -2.54]$	10/20
Qwen-Max API	10/10; 10/10; 9/10	8	-5.70 $[-9.18, -2.49]$	-0.47 $[-1.18, 0.44]$	10/20

What this does not test. Whether a procedural remedy and a team remedy compose. That needs a factorial crossing procedural retrieval (on/off) with team context (on/off) on tasks that require both a deep dependency chain and a discovery handoff, which we have not run. A separate 120-trial allocation benchmark that touches the work-ledger path is not reported here and carries no claim.

References

Anthropic. Claude Sonnet 4.5. `https://www.anthropic.com/news/claude-sonnet-4-5`, 2025.

Gautier Dagan, Frank Keller, and Alex Lascarides. Plancraft: An evaluation dataset for planning with LLM agents. In *Proceedings of the 2nd Conference on Language Modeling (COLM)*, 2025.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.

Robert G. Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17(22):2635–2650, 1998.

OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yi Xu, Philipp Jettkant, and Laura Ruis. The depth ceiling: On the limits of large language models in discovering latent planning. In *Proceedings of the 3rd Conference on Language Modeling (COLM)*, 2026.